



Map Topology and Strategic Emergence in Risk: A Reinforcement Learning Approach

Final Project Report

Author: Ahmer Alam

Supervisor: Dr David Kohan Marzagao

Student ID: K22008566

April 21, 2026

Abstract

Strategic dominance in the board game: *Risk* is shaped not only by player skill, but by the map itself. This work examines how topological properties such as connectivity, density, and bottleneck formation influence strategic behaviour and overall game balance in Risk-style environments through the integration of reinforcement learning and graph-theoretic modelling. A formal map representation allows any Risk map to be instantiated as a playable environment within a consistent agent-environment framework. On this basis, a reinforcement learning agent trained using Proximal Policy Optimisation (PPO) is evaluated and compared against a set of heuristic agents designed to represent distinct tactical behaviours. Controlled experiments analyse how structural variation shapes strategic performance, aggression dynamics, learning stability, and balance across configurations. The resulting framework provides a modular system for reproducible experimentation across different map configurations and agent compositions, enabling systematic study of multi-agent strategic behaviour in complex environments.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Ahmer Alam

April 21, 2026

Acknowledgements

I would like to thank my project supervisor, Dr David Kohan Marzagao, for his guidance, feedback, and support throughout the duration of this project. His encouraging words were a great source of motivation, and his advice was invaluable in shaping both the direction and development of this work.

Contents

1	Introduction	2
1.1	Project Aims and Objectives	3
2	Background and Context	4
2.1	The Agent and the Environment	4
2.2	Markov Decision Processes (MDPs)	5
2.3	Deep Neural Networks	7
3	Literature Review	11
3.1	Existing Deep Reinforcement Learning Algorithms	11
3.2	Existing Battle Probability Matrix Computations	20
4	Design and Implementation	22
4.1	Eliciting the Metadata of a Risk Map	22
4.2	Instantiating a Map-agnostic Risk Environment	24
4.3	Declaring Heuristic Agents	30
4.4	Training the RL Agent	34
5	Experiments	39
5.1	Experiment 1: Effect of Map Density on Concentrated vs Distributed Strategies	39
5.2	Experiment 2: Effect of Map Size on First-Mover Advantage in Two-Player Risk	45
5.3	Experiment 3: Qualitative Behavioural Study of the RL Agent	49
6	Conclusion and Future Work	54
6.1	Conclusion	54
6.2	Future Work	55
7	Legal, Social, Ethical and Professional Issues	56
7.1	Legal Issues	56
7.2	Social and Ethical Issues	57
7.3	Professional Issues	57
	Bibliography	60

Chapter 1

Introduction

In the realm of game theory, non-determinism refers to systems where the same actions can lead to different outcomes across runs [1]. In deterministic board games such as Chess or Go, a given sequence of moves will always lead to the same outcome, allowing strategic reasoning to be based entirely on foresight. In contrast, many real-world games are inherently non-deterministic, meaning that chance plays a fundamental role in shaping game dynamics and outcomes. This non-determinism introduces uncertainty, making optimal decision-making significantly more challenging and less amenable to exhaustive planning.

Risk is a classical example of a non-deterministic board game. Players compete for global domination by conquering territories, forming continents, and managing limited resources, all while contending with stochastic combat outcomes driven by dice rolls and the random acquisition of territory cards. A particularly influential yet under-explored factor in Risk is the structural design of the game map itself. Different maps vary in size, connectivity, choke points, and continent structures, which can significantly affect the viability of certain strategies. For example, a Gung-ho playstyle that aggressively and greedily engages in frequent battles may yield considerable success on one map, yet translate poorly to another.

While human players often develop intuitive heuristics through experience, explicitly eliciting effective strategies for a computer-controlled agent remains challenging. The complexity of strategic decision-making, combined with uncertainty and delayed consequences, makes it difficult to rely solely on manually designed rules. Reinforcement learning offers an alternative approach in which agents improve behaviour through repeated interaction with an environment.

1.1 Project Aims and Objectives

This project aims to investigate how strategic behaviour in the board game Risk emerges and varies as a function of the underlying map topology, in order to analyse the relationship between map structure, strategic effectiveness, and broader notions of game balance.

This aim can be broken down into a sequence of objectives as follows:

1. Map modelling: To identify a set of abstract structural properties capturing the essential characteristics of Risk maps, and to generate concrete map instances from these abstractions.
2. Environment design: To develop a map-agnostic Risk environment that formalises the rules, state and action spaces, and transition dynamics independently of any specific map topology.
3. Agent development: To create heuristic baseline agents and use them to train reinforcement learning agents across different map instances, until stable strategic behaviour is observed.
4. Experimental analysis: To evaluate learned and baseline heuristic strategies across map configurations in order to analyse how changes in map structure influence strategic outcomes, agent behaviour, and overall game balance.

The following assumptions will be made for this project:

- Unless explicitly stated otherwise, this project will adhere to the Hasbro Risk ruleset [2].
- While multiple variants of Risk exist, this project focuses specifically on the *Risk: Global Domination* format.

Chapter 2

Background and Context

2.1 The Agent and the Environment

In 1998, Richard Sutton and Andrew Barto fundamentally shifted the landscape of AI with their book, *Reinforcement Learning: An Introduction* [3]. They took scattered ideas from animal psychology and control theory and fused them into a single, powerful framework, giving the world a standardised "playbook" for how machines can learn from experience.

In the book, they argue that the real power of reinforcement learning lies in its abstraction: any sequential decision-making problem can be framed as a simple interaction between an agent and an environment. Once such reduction has been done, a wide range of established RL algorithms can be applied directly to learn effective behaviour, thereby offering general-purpose solution methods for the original task.

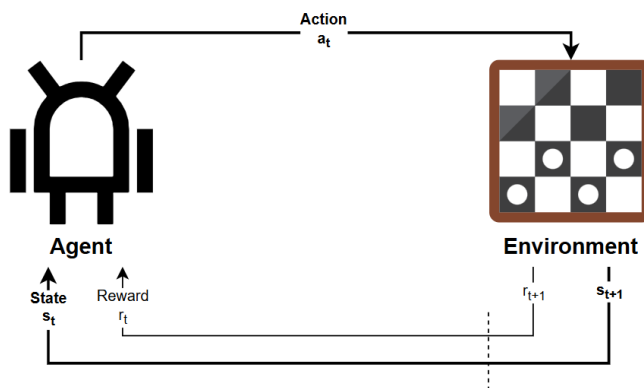


Figure 2.1: The agent-environment interaction in reinforcement learning.

An **agent** is simply an entity capable of learning and making decisions. This is done by interacting with an **environment**, which represents everything outside of the agent’s direct control.

At a particular point in time t , the agent receives a representation of the environment’s **state** $s_t \in S$ such that S denotes the set of all possible states. It then selects an **action** $a_t \in A(S)$ such that $A(S)$ denotes the set of all possible actions available from state S . Finally, the environment interprets a_t in order to return a new state s_{t+1} , in addition to a numerical **reward** $r_{t+1} \in \mathbb{R}$ that denotes the immediate consequence of taking that action; the higher the reward the better. This cycle continues until the agent reaches a terminal state $s \in S_{end} \subset S$ (such that S_{end} denotes the set of all possible terminal states), marking the end of an **episode**. The sequence of states, actions, and rewards for a particular episode $(s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{end})$ is referred to as a **trajectory**.

2.2 Markov Decision Processes (MDPs)

Observing the modelling of the agent-environment interaction, the next state s_{t+1} and reward r_{t+1} depend only on the current state s_t and action a_t ; the full trajectory history is not needed. This property is known as **Markov Property**, and must be satisfied in order to utilise MDPs.

A **Markov Decision Process** mathematically formalises this sequential-decision making problem in which a single agent interacts with an environment. MDPs are inherently **stochastic**: the outcome of an action is not fixed but determined by a transition function that specifies the probability of moving to each possible next state. This characteristic aligns naturally with the non-deterministic elements of the game of Risk, where, for example, the outcome of an attack is resolved through dice rolls and therefore cannot be predicted with certainty.

MDPs are formally defined by the following tuple:

$$MDP = (S, A, T, R, \gamma)$$

- S = State space (i.e. set of all possible states).
- A = Action space (i.e. set of all possible actions).
- T = Transition function : $S \times A \times S \rightarrow [0, 1]$. $T(s, a, s')$ denotes the probability of reaching

state s' after performing action a at state s . It follows that $\sum_{s' \in S} T(s, a, s') = 1$.

- R = Reward function : $S \times A \rightarrow \mathbb{R}$. $R(s, a)$ denotes the immediate reward after performing action a at state s .
- γ = Discount factor $\in [0, 1]$, determines how the agent balances the importance of immediate rewards (low γ) and future rewards (high γ).

2.2.1 Solving an MDP

An agent's behaviour is defined by its **policy**. Formally, a policy is a mapping function $\pi : S \times A \rightarrow [0, 1]$ such that $\pi(a|s)$ denotes the probability of the agent performing action a given that it is at state s . The solution to an MDP is to identify an **optimal policy** (π^*) which selects the best actions for the agent by maximising its expected return. This objective can be broken down into the following sub-tasks:

1. At time t , the **return** ($G_t \in \mathbb{R}$) refers to the sum of all immediate rewards from time t onwards along its trajectory, until the end of the episode at time T :

$$G_t = R(s_t, a_t) + \gamma R(s_{t+1}, a_{t+1}) + \gamma^2 R(s_{t+2}, a_{t+2}) + \dots + \gamma^{T-t} R(s_T, a_T) = \sum_{k=0}^{T-t} \gamma^k R(s_{t+k}, a_{t+k})$$

2. The **state-value function** ($V^\pi(s) : S \rightarrow \mathbb{R}$) refers to the expected return given that the agent begins at state s and follows policy π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

The **action-value function** ($Q^\pi(s, a) : S \times A \rightarrow \mathbb{R}$) is an extension which also considers the action a performed at state s :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

As a reminder, the return G_t is a random variable: even if the agent begins at the same state and takes the same action, the stochastic nature of the policy can lead to different trajectories and thus different rewards. Therefore, value functions typically weigh over each return by their probability, a.k.a the expectation ($\mathbb{E}$) of the return.

3. For finite MDPs, there always exists at least one deterministic policy that is optimal.

Such an optimal policy π^* satisfies the following properties:

- $\forall \pi, \forall s \in S, V^{\pi^*}(s) \geq V^\pi(s)$
- $\forall \pi, \forall s \in S, \forall a \in A, Q^{\pi^*}(s, a) \geq Q^\pi(s, a)$

Computing such optimal policy efficiently remains one of the central challenges in reinforcement learning; relevant approaches are introduced in the following section. The key takeaway is that the action-value function for the optimal policy $Q^{\pi^*}(s, a)$ is sufficient for optimising the behaviour of an agent in an MDP: out of all actions for a particular state, the optimal policy selects the one with the highest action-value.

2.3 Deep Neural Networks

Artificial **neural networks** are machine learning models designed to learn complex relationships between inputs and outputs by processing data through layers of interconnected neurons [4]. A typical neural network is composed of three main types of layers: an input layer, one or more hidden layers, and an output layer. The input layer represents the features or observations provided to the model, while the output layer produces the final prediction or decision of the network. Between these layers lie the hidden layers, which progressively transform the input information into increasingly abstract representations. When a neural network contains multiple hidden layers, it is referred to as a **deep neural network**. Increasing the depth of the network enables it to learn hierarchical representations of data, allowing complex patterns to be expressed as compositions of simpler transformations [5].

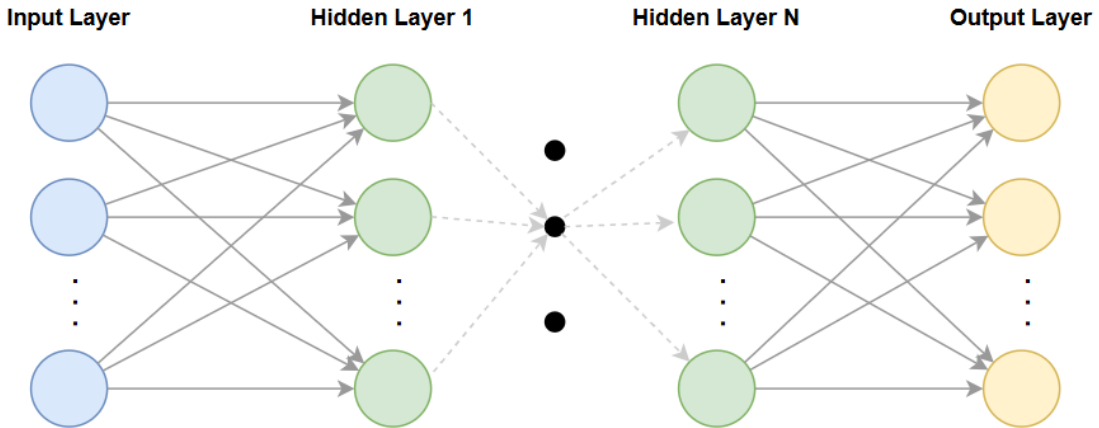


Figure 2.2: Schematic of a feed-forward deep neural network.

2.3.1 Encoding a Neural Network

A deep neural network θ with $L > 1$ hidden layers is parametrised by a sequence of weight matrices, $\theta = (\mathbf{W}^{I=0}, \mathbf{W}^1, \dots, \mathbf{W}^{L-1}, \mathbf{W}^{O=L})$. Matrices $\mathbf{W}^I$ and $\mathbf{W}^O$ represent the input-to-hidden and hidden-to-output transitions respectively. For any given layer l , the matrix $\mathbf{W}^l \in \mathbb{R}^{j \times (i+1)}$ stores the connection weights between the i neurons in layer l and the j neurons in layer $l + 1$. Specifically, w_{ji} represents the weight to neuron j in the succeeding layer from neuron i in the current layer, with the first column ($i = 0$) reserved for the bias terms:

$$\mathbf{W}^l = \begin{pmatrix} w_{10} & w_{11} & w_{12} & \cdots & w_{1i} \\ w_{20} & w_{21} & w_{22} & \cdots & w_{2i} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{j0} & w_{j1} & w_{j2} & \cdots & w_{ji} \end{pmatrix} \in \mathbb{R}^{j \times (i+1)}$$

2.3.2 Computing Neuron Activations

The neurons within each layer store a scalar value known as the **activation**. The activations of the input layer, $a^{(0)}$, are provided externally and represent a vectorized encoding of the problem (e.g. the state of the board in a game of Risk using multiple neurons). Conversely, the activations of the output layer, $a^{(L)}$, represent the target entity the network is trained to approximate (e.g. the optimal action to take from a given game state, using a single neuron). For any hidden or output layer, the activations are computed by applying an activation function to the weighted sum of the previous layer's activations:

$$a_j^{(l+1)} = \sigma \left(net_j = \sum_{i=0}^n w_{ji}^{(l)} a_i^{(l)} \right)$$

It follows that σ , the **activation function**, is non-linear. This non-linearity is critical; without it, the entire neural network would collapse into a single linear transformation, rendering it incapable of learning complex, non-linear patterns such as those found in strategic game states. Common choices for σ include Rectified Linear Unit or ReLU ($\sigma(x) = \max(0, x)$), Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, Hyperbolic Tangent ($\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$). Selecting the appropriate activation function depends on the layer's role within the architecture; for instance, ReLU is typically preferred for hidden layers due to its ability to keep the training of weights fast, while Sigmoid or Tanh are often utilized when the output must be constrained to a specific range.

2.3.3 Training a Neural Network

Training a neural network is an iterative optimisation process aimed at minimizing a **Loss Function**, $J(\hat{y}, y)$, which quantifies the discrepancy between the network’s prediction $\hat{y}$ and the ground-truth target y .

The primary mechanism for learning is **Backpropagation**. This algorithm calculates the gradient of the loss function, ∇J , with respect to every weight in the network. The gradient is a vector pointing in the direction of the steepest *ascent* in the loss landscape. Consequently, to minimise error, the weights must be updated by moving in the opposite direction of the gradient. **Stochastic Gradient Descent (SGD)** facilitates this by iteratively ”stepping” down the loss surface toward a local minimum. The update rule for a specific weight is:

$$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial J}{\partial w_{ji}^{(l)}}$$

Hyperparameter Tuning

While parameters (weights and biases) are the internal values learned by the network during training, **hyperparameters** are the external configurations set by the practitioner before training begins. Finding optimal hyperparameter values is critical since they dictate the efficiency, speed, and stability of the learning process. Common neural network hyperparameters are:

- **Learning Rate (η):** Controls the step size during gradient descent. An excessively high rate can cause the model to overshoot the minimum and diverge, while a rate that is too low results in slow convergence and a higher risk of stalling in local minima.
- **Batch Size:** The number of training samples processed per gradient update. Larger batches offer stable gradient estimates, whereas smaller batches introduce stochastic ”noise” that can help the network escape sharp local minima.
- **Epochs:** The total number of complete passes through the training dataset. While more epochs provide more refinement, an excessive count often leads to **overfitting**, where the model memorises training noise rather than generalising to unseen data.
- **Activation Function:** Choosing functions like ReLU over Sigmoid helps mitigate the vanishing gradient problem, ensuring that weights in the earliest layers receive sufficient updates during backpropagation [6].

Parameter Tractability and Computational Complexity

The "deep" in deep learning refers to the accumulation of layers, which leads to a massive growth in the number of trainable parameters. The total number of parameters P is:

$$P = \sum_{l=0}^{L-1} (n_l + 1)n_{l+1}$$

where n_l is the number of neurons in layer l , and the $+1$ accounts for the bias terms. In a game like Risk, where the input vector must represent dozens of territories and troop counts, even a modest architecture can result in hundreds of thousands of parameters. This high dimensionality makes the loss surface extremely complex, requiring significant computational power and training time to converge on an effective strategy.

2.3.4 Applying Neural Networks in Reinforcement Learning

While the fundamental objectives of Reinforcement Learning (RL) revolve around identifying an optimal policy π^* through the estimation of value functions like $V^\pi(s)$ or $Q^\pi(s, a)$, traditional tabular methods become computationally intractable as the state space S grows. In a complex environment such as Risk, the number of possible board configurations is astronomically high, making it impossible to store and update a unique value for every state. In the modern era, nearly all high-performance RL algorithms address this "curse of dimensionality" [7] by utilizing neural networks as powerful non-linear function approximators.

This integration, often referred to as **Deep Reinforcement Learning (DRL)**, shifts the focus from maintaining discrete tables to learning the parameters of a network that can generalise across similar states. While the subsequent chapter will detail specific DRL algorithms, it is important to note that they differ primarily in how they leverage the neural network architecture; these variations include the choice of loss function, the functional purpose the network serves within the learning loop, or even the total number of neural networks utilised.

Chapter 3

Literature Review

3.1 Existing Deep Reinforcement Learning Algorithms

Deep Reinforcement Learning (DRL) algorithms seek an optimal policy π^* that maximizes the expected return G_t within an MDP framework. These approaches are broadly categorized into model-based and model-free methods. **Model-based** methods leverage or learn the transition function $T(s, a, s')$ and reward function $R(s, a)$ to simulate future states, enabling the agent to plan or derive a policy through internal prediction. Conversely, **model-free** methods skip explicit estimation of T and R , instead learning decision-making strategies directly from experience by estimating the state-value function $V^\pi(s)$ and action-value function $Q^\pi(s, a)$ [3].

A central challenge in these estimations is the **bias-variance trade-off**. When approximating quantities such as G_t or $Q^\pi(s, a)$, the agent ideally seeks estimates with both low bias (small systematic error) and low variance (small fluctuations across different sampled trajectories). In practice, however, reducing one often increases the other [8]. Parallel to this is the **exploration-exploitation trade-off**: the agent must choose between exploiting known high-reward actions and exploring new strategies to ensure the policy π doesn't converge on a local optimum [9].

The following subsections review several influential DRL approaches, highlighting their algorithmic foundations, notable applications in the literature, and their relative suitability for learning policies for the board game Risk.

3.1.1 Monte-Carlo Tree Search (MCTS)

Monte-Carlo Tree Search (MCTS) is an iterative model-based search algorithm designed to make decisions in large sequential decision problems by performing stochastic simulations of possible future trajectories. Early variants of Monte-Carlo search appeared in computer Go programs in the mid-2000s, where traditional minimax search struggled due to the enormous branching factor of the game [10]. In 2016, the technique gained significant attention following the success of AlphaGo, which combined MCTS with deep neural networks to defeat professional human Go players [11]. Subsequent systems such as AlphaGo Zero and AlphaZero further demonstrated the effectiveness of combining deep learning with tree search to achieve superhuman performance in games including Go and Chess [12].

Algorithmic Foundations

MCTS constructs a search tree rooted at the current state and iteratively expands it through four stages: selection, expansion, simulation, and backpropagation. During selection, the algorithm traverses the existing tree by choosing actions that balance exploration and exploitation. A commonly used rule for this is the **Upper Confidence Bound (UCB)** criterion:

$$a^* = \arg \max_a \left(Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right)$$

where $Q(s, a)$ represents the estimated value of taking action a at state s , $N(s)$ denotes the number of times state s has been visited, $N(s, a)$ is the number of times action a has been selected from s , and c is a hyperparameter controlling the exploration–exploitation balance. If an action has not yet been explored ($N(s, a) = 0$), its UCB value is treated as ∞ , ensuring it will be selected next. The tree is then expanded by adding the resulting successor state. From this new state, the remainder of the episode is simulated (a **rollout**) to estimate the outcome of the trajectory, and the resulting reward is subsequently propagated back through the tree to update the value estimates of previously visited nodes.

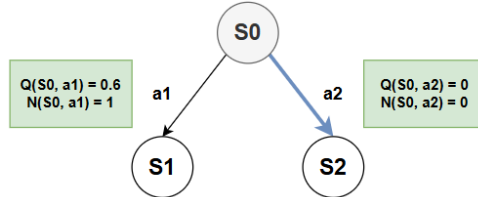


Figure 3.1: A snapshot of a Monte-Carlo tree. The action $a2$ has $UCB(a2) = \infty$ because it has not yet been explored ($N(s0, a2) = 0$), making it the next action selected in this iteration.

Modern implementations often integrate deep neural networks into this process. Instead of relying purely on random rollouts, a separate neural network can be trained to approximate a policy $\pi(a|s)$ and a value function $V(s)$. The policy network guides which actions should be explored during tree search, while the value network provides an estimate of the expected return from a given state. This combination allows MCTS to focus simulations on more promising regions of the search space and significantly improves decision quality.

Suitability for Risk

MCTS has previously been applied to Risk in academic research. Blomqvist implements an AlphaZero-style framework in which Monte Carlo Tree Search is combined with neural network policy and value estimation [13]. Notably, rather than employing a single unified policy over the entire action space, the approach decomposes the game into distinct phases and trains separate policies tailored to each phase. This modular design reduces the effective action space per decision and aligns the learning problem with the structural decomposition of Risk turns. Irrespective of the specific reinforcement learning algorithm employed, such a phase-specific architectural strategy remains a viable design choice. However, the variable action spaces introduced by differing map configurations would considerably increase implementation complexity, placing this extension beyond the scope of the present experimental study.

A common challenge when applying MCTS to Risk is the extremely long episodes that arise during gameplay. While a typical game of Go consists of roughly 200–300 actions [14], the granular mechanics of Risk, including troop deployment, repeated combat decisions, and fortification, can extend a single game to tens of thousands of actions. This depth, combined with a branching factor that grows with map size, renders exhaustive tree exploration computationally impractical. Prior work has attempted to mitigate this through dynamic control of the branching factor and action pruning. The authors of [15] adjust the set of considered actions depending on the current phase of the game and prune strategically irrelevant moves to restrict tree expansion. Such mechanisms partially alleviate computational complexity by focusing simulations on a reduced subset of promising actions. However, accurately determining which actions can be safely pruned without degrading long-term strategic evaluation remains challenging in a stochastic, long-horizon environment such as Risk.

A more fundamental limitation of MCTS for the purposes of this project is its dependence on extensive online simulations for action evaluation. Unlike model-free methods, which select actions using a single forward pass through a neural network, MCTS typically requires thousands of rollouts per decision to obtain reliable value estimates. This significantly increases the computation required during inference. In the context of this project, such high “thinking time” per move would considerably slow experimentation: either experiments would take an unacceptably long time to run, or the number of simulations per move would need to be reduced, potentially weakening the reliability of the results.

3.1.2 Deep Q-Networks (DQN)

Introduced in 2015 by researchers at DeepMind, **Deep Q-Networks (DQN)** demonstrated that deep neural networks could learn effective control policies directly from high-dimensional visual observations. The system was evaluated on Atari 2600 video games (a diverse collection of arcade-style games including Pac-Man and Space Invaders), and achieved human-level performance across many titles using a single, general architecture [16]. This work provided strong empirical evidence that neural networks could approximate value functions in complex environments without manual feature engineering. The success of DQN directly motivated subsequent research combining neural network function approximation with other decision-making frameworks, including the integration of neural networks into search-based methods such as MCTS as seen in the previous section.

Algorithmic Foundations

Q-learning is a model-free reinforcement learning technique that seeks to learn the optimal action-value function $Q^*(s, a)$, which represents the expected return of taking action a in state s and following the optimal policy thereafter [17]. The core update rule is based on the **Bellman Optimality Equation**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{New Estimate}} - \underbrace{Q(s, a)}_{\text{Old Estimate}} \right]$$

where α is the learning rate, r is the reward obtained after taking action a in state s , and s' is the next state. The term inside the brackets is the **Temporal Difference Error**, and nudges the current estimate towards the newly observed target. By iteratively applying this update over sampled transitions, Q-learning converges to the optimal Q-function.

In traditional Q-learning, the output is a Q-Table, $Q^* \in \mathbb{R}^{|S| \times |A|}$, such that $\pi^*(s) = \arg \max_{a \in A} Q^*(s, a)$. However, enumerating such a table for complex state spaces like Risk is computationally infeasible due to the aforementioned "curse of dimensionality". Deep Q-Networks address this by using a neural network as a non-linear function approximator, $Q(s, a; \theta)$. Instead of storing discrete values, the network learns to generalise from seen states to unseen ones, outputting Q-value estimates for all possible actions given a state input.

To ensure stability during training, DQN introduces two critical mechanisms: experience replay and target networks. Experience replay stores transitions (s, a, r, s') in a memory buffer D , allowing the agent to sample random "batches" of past experiences, in turn preventing the network from overfitting to recent trends. Additionally, a separate target network with parameters θ^- is maintained to compute the target value in the loss function, preventing the target from changing too rapidly as the main network is updated. Putting this all together, the network is optimised by minimising the mean squared temporal difference error, where the loss function backpropagated to update the parameters θ is:

$$J(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta^-)}_{\text{New Estimate}} - \underbrace{Q(s, a; \theta)}_{\text{Old Estimate}} \right)^2 \right]$$

Suitability for Risk

When considering Risk, DQNs offer some encouraging advantages. Being model-free, they do not require knowledge of the transition function, and once trained they can select actions efficiently through a single forward pass of the network. This enables relatively fast inference compared to planning-based methods such as MCTS. Furthermore, the neural network can generalise across similar states, potentially allowing the agent to capture strategic patterns within the game.

A notable challenge highlighted in similar non-deterministic board game domains is the sensitivity of DQN to reward scaling. In their study on Ticket to Ride, Strömberg and Lind observed that agents trained with small, normalised floating-point rewards initially performed poorly compared to agents using larger integer rewards, requiring substantially more experience to reach comparable performance [18]. While the specific reward values are domain-dependent, this finding suggests that poorly scaled rewards in Risk could similarly slow early learning or destabilize Q-value updates. Careful design of the reward function R , including consideration

of magnitude and sparsity, is therefore crucial when applying DQN to complex, long-horizon games like Risk. Despite these challenges, DQNs remain a viable approach for learning policies in Risk and share several design principles with the approach presented in the following section.

3.1.3 Proximal Policy Optimisation (PPO)

In 2017, researchers at OpenAI introduced **Proximal Policy Optimisation (PPO)** as a reinforcement learning algorithm designed to improve the stability and reliability of policy-gradient methods (i.e. learning $\pi_\theta(a|s)$ directly) [19]. Earlier policy optimisation approaches often suffered from unstable learning when policy updates became too large, leading to sudden performance degradation during training. PPO addressed this by constraining how much the policy can change between updates, allowing learning to proceed through a series of smaller, more stable improvements while remaining simple to implement with standard gradient-based optimisation. Due to its balance between performance, stability, and computational efficiency, PPO has since become one of the most widely adopted algorithms in deep reinforcement learning and is widely used in Reinforcement Learning from Human Feedback (RLHF) pipelines for training Large Language Models (LLMs) such as ChatGPT [20].

Algorithmic Foundations

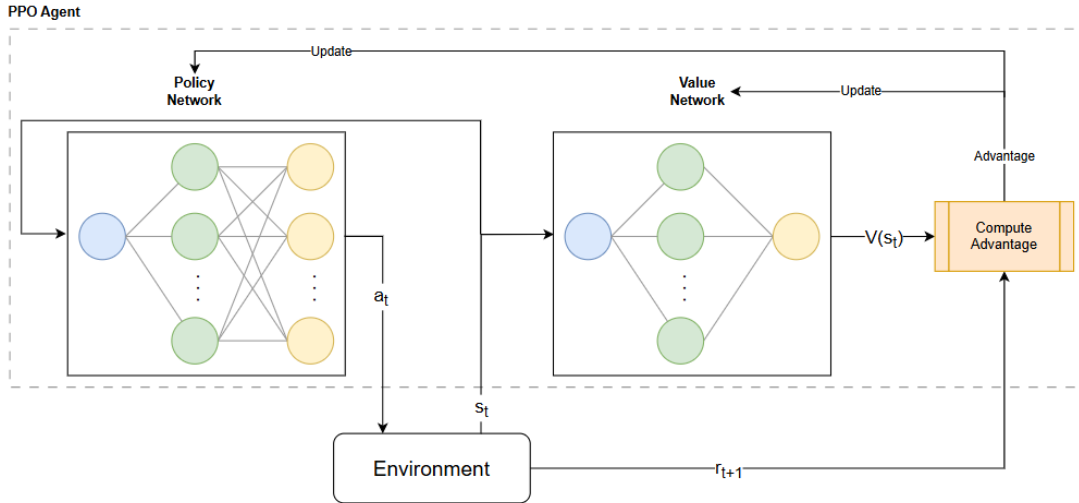


Figure 3.2: Overview of the PPO actor-critic architecture. The policy network has $|A|$ output neurons representing the probability of selecting each action, while the value network has a single output neuron producing the state-value estimate $V(s)$.

PPO is implemented using an **actor-critic architecture**, in which two neural networks are trained simultaneously. The actor represents the policy network $\pi_\theta(a|s)$, which maps a state s to a probability distribution over possible actions. The critic is a value network $V_\phi(s)$ that

estimates the expected return from a given state, producing a scalar prediction of the long-term reward achievable from that state. This value estimate acts as a baseline that evaluates how good a chosen action was relative to expectation. In this sense, the critic plays a similar role to the DQN seen previously, as it approximates the expected return of states, but instead of selecting actions directly it guides updates to the policy network during training.

To evaluate the quality of an action, PPO uses an **advantage function**, which measures how much better or worse the observed outcome of it was compared to expectation:

$$A_t = G_t - V_\phi(s_t)$$

The value network is trained to approximate the state-value function $V_\phi(s)$, which estimates the expected return from a given state. It is optimised by minimising the mean squared error between the predicted value and the observed return over a batch of data. This encourages the critic to produce accurate value estimates, which in turn lead to more reliable advantage calculations and help stabilise updates to the policy network.

$$J(\phi) = \mathbb{E}_t [(V_\phi(s_t) - G_t)^2]$$

The loss function of the policy network is:

$$J(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

The **probability ratio**, $r_t(\theta)$, measures how the chance of selecting action a_t has changed under the updated policy compared to the previous policy (whose parameters θ_{old} are fixed during the update step). If $r_t(\theta) > 1$, the new policy increases the likelihood of taking that action, whereas $r_t(\theta) < 1$ indicates a decrease. Multiplying this ratio by the advantage A_t encourages the policy to increase the probability of advantageous actions ($A_t > 0$) and decrease the probability of disadvantageous ones ($A_t < 0$):

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

However, directly optimising $r_t(\theta)A_t$ can lead to excessively large policy updates. To address this, PPO introduces a **clipping function** that restricts the ratio to the interval $[1 - \epsilon, 1 + \epsilon]$. If the ratio moves outside this range, it is truncated to the nearest bound, preventing further amplification of the update. The final objective takes the minimum between the unclipped and clipped terms, ensuring that updates which would push the policy too far are suppressed. This clipping mechanism stabilises training by limiting abrupt changes in the policy, allowing learning to proceed through smaller, more controlled improvements.

Suitability for Risk

PPO effectively addresses the structural bottlenecks of both MCTS and DQN, making it a robust choice for the Risk environment. Unlike MCTS, which is hindered by the long planning horizon of tens of thousands of sequential actions, PPO is a model-free method that learns a direct mapping from states to actions. By representing the policy with a neural network, PPO removes the need for computationally expensive online simulations at inference time, enabling efficient decision-making via a single forward pass.

PPO is generally more robust to reward scaling than value-based methods such as DQN. Its clipped objective function constrains policy updates within a bounded range defined by the hyperparameter ϵ , preventing overly large updates, while the use of a normalised advantage estimates mitigates the impact of reward magnitude on learning dynamics. Together, these mechanisms allow PPO to learn stable policies without the careful reward scaling that DQN requires, making it well-suited for environments with complex, sparse, or delayed rewards like Risk.

Invalid Action Masking in Policy Gradient Methods

In Risk, a player may choose to initiate a battle by selecting a pair of territories to attack from and to. When enumerating this, there are a quadratic number of possible actions to represent in the output layer of the neural network: $42 \times 42 = 1764$ battle actions for the Earth map alone! However, due to the constraints imposed by the game (the source territory must be owned by the player with at least two troops, the target territory must be owned by an opponent, and the territories must be adjacent), the vast majority of these battle actions are invalid in any given state.

A naive approach to handling invalid actions is to penalise the agent with a negative reward whenever such an action is selected. While simple to implement, this strategy does not scale well in environments with large and sparse action spaces. Since invalid actions can vastly outnumber valid ones, the agent may spend a significant portion of training exploring and being penalised for illegal moves, leading to inefficient learning and slow convergence. Moreover, the penalty signal does not explicitly inform the agent which actions are valid, making it difficult to reliably learn the underlying constraints of the environment.

A more effective approach is **invalid action masking**, as explored by Huang and Ontañón [21]. Instead of penalising invalid actions after they are selected, masking removes them from consideration entirely during action selection. This approach scales significantly better in large action spaces, as the agent is restricted to exploring only feasible decisions at each timestep. As a result, learning becomes more sample-efficient, faster to converge, and more stable, particularly in environments such as Risk where the set of valid actions changes dynamically and constitutes only a small subset of the full action space.

Invalid action masking is incorporated into modern PPO via the **MaskablePPO** framework, developed by Stable-Baselines3 [22].

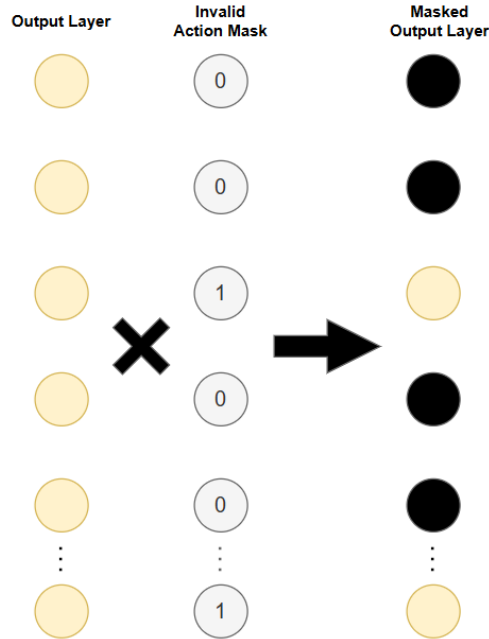


Figure 3.3: The invalid action masking mechanism. A binary mask is applied to the raw output of the policy network to zero out illegal actions.

3.2 Existing Battle Probability Matrix Computations

A key component of the Risk environment lies in accurately modelling the state transitions induced by **battles**. Given a battle in which an attacker with A troops attacks a defender with D troops, there are two fundamental questions:

1. What is the probability that the attacker wins the battle?
2. What is the probability distribution over the remaining troops of the attacker and defender once the battle concludes?

This web application provides a heuristic estimate of battle outcomes by repeatedly simulating an (A/D) attack and reporting the empirical win probability [23]. The resulting estimates are non-deterministic and vary across identical runs.

Osbourne adopts a more theoretical approach, leveraging linear algebra and Markov chains in order to compute a "Transition Probability Matrix" [24]. In this matrix, the rows and columns correspond to the initial attacker and defender troop counts A and D respectively, and each entry gives the exact probability of the attacker winning the battle.

While several publications, including those cited above, offer comprehensive answers to the first question, significantly less attention has been paid in describing *how* the battle is won/lost (Q2). Specifically, given an (A/D) battle, what is the probability that the attacker and defender are left with R_A and R_D troops respectively?

$$P((R_A/R_D) \mid (A/D))$$

This repository contains the Risk dice engine implemented in C# by SMG Studio, the official developers of *Risk: Global Domination* [25]. Its accompanying documentation describe how to simulate battles in the true random blitz format for arbitrary (A/D) configurations. Furthermore, the engine computes the arrays **AttackLossChances** and **DefendLossChances**, which together encode the full distribution over possible remaining troop counts R_A and R_D . These simulations were exhaustively executed across a broad range of battle configurations in order to compute the **Blitz Probability Matrix** (Ω). A subset of the resulting probabilities (up to 6 versus 6 battles) are as follows:

$(A/D) \backslash (R_A/R_D)$	(6/0)	(5/0)	(4/0)	(3/0)	(2/0)	(1/1)	(1/2)	(1/3)	(1/4)	(1/5)	(1/6)	Win
(6/6)	0.051	0.092	0.123	0.087	0.043	0.060	0.117	0.134	0.124	0.104	0.064	0.397
(6/5)	0.091	0.124	0.147	0.096	0.049	0.068	0.134	0.124	0.104	0.064	0.000	0.506
(6/4)	0.138	0.165	0.179	0.107	0.050	0.070	0.124	0.104	0.064	0.000	0.000	0.638
(6/3)	0.245	0.208	0.175	0.097	0.045	0.062	0.104	0.064	0.000	0.000	0.000	0.769
(6/2)	0.372	0.222	0.184	0.079	0.033	0.046	0.064	0.000	0.000	0.000	0.000	0.890
(6/1)	0.660	0.224	0.076	0.023	0.007	0.010	0.000	0.000	0.000	0.000	0.000	0.990
(5/6)	0.000	0.051	0.092	0.071	0.039	0.054	0.119	0.133	0.166	0.144	0.131	0.253
(5/5)	0.000	0.091	0.124	0.096	0.048	0.067	0.133	0.166	0.144	0.131	0.000	0.359
(5/4)	0.000	0.138	0.165	0.115	0.059	0.083	0.166	0.144	0.131	0.000	0.000	0.477
(5/3)	0.000	0.245	0.208	0.129	0.060	0.083	0.144	0.131	0.000	0.000	0.000	0.642
(5/2)	0.000	0.372	0.222	0.133	0.060	0.083	0.131	0.000	0.000	0.000	0.000	0.785
(5/1)	0.000	0.660	0.224	0.067	0.020	0.028	0.000	0.000	0.000	0.000	0.000	0.972
(4/6)	0.000	0.000	0.051	0.053	0.029	0.041	0.090	0.135	0.176	0.206	0.218	0.134
(4/5)	0.000	0.000	0.091	0.073	0.042	0.059	0.135	0.176	0.206	0.218	0.000	0.206
(4/4)	0.000	0.000	0.138	0.116	0.060	0.085	0.176	0.206	0.218	0.000	0.000	0.315
(4/3)	0.000	0.000	0.245	0.150	0.075	0.106	0.206	0.218	0.000	0.000	0.000	0.470
(4/2)	0.000	0.000	0.372	0.194	0.090	0.126	0.218	0.000	0.000	0.000	0.000	0.656
(4/1)	0.000	0.000	0.660	0.197	0.060	0.084	0.000	0.000	0.000	0.000	0.000	0.916
(3/6)	0.000	0.000	0.000	0.012	0.010	0.013	0.041	0.071	0.164	0.242	0.448	0.021
(3/5)	0.000	0.000	0.000	0.030	0.019	0.027	0.071	0.164	0.242	0.448	0.000	0.049
(3/4)	0.000	0.000	0.000	0.052	0.039	0.055	0.164	0.242	0.448	0.000	0.000	0.091
(3/3)	0.000	0.000	0.000	0.132	0.074	0.104	0.242	0.448	0.000	0.000	0.000	0.206
(3/2)	0.000	0.000	0.000	0.228	0.135	0.189	0.448	0.000	0.000	0.000	0.000	0.363
(3/1)	0.000	0.000	0.000	0.579	0.176	0.246	0.000	0.000	0.000	0.000	0.000	0.754
(2/6)	0.000	0.000	0.000	0.000	0.000	0.001	0.003	0.012	0.048	0.190	0.745	0.000
(2/5)	0.000	0.000	0.000	0.000	0.002	0.002	0.012	0.048	0.190	0.745	0.000	0.002
(2/4)	0.000	0.000	0.000	0.000	0.007	0.010	0.048	0.190	0.745	0.000	0.000	0.007
(2/3)	0.000	0.000	0.000	0.000	0.027	0.038	0.190	0.745	0.000	0.000	0.000	0.027
(2/2)	0.000	0.000	0.000	0.000	0.106	0.149	0.745	0.000	0.000	0.000	0.000	0.106
(2/1)	0.000	0.000	0.000	0.000	0.417	0.583	0.000	0.000	0.000	0.000	0.000	0.417

The row $\Omega_{(A/D)}$ contains the remaining troop probability distributions of the battle. For example, the entry $\Omega_{(5/1),(4/0)} = 0.224$, indicates that there is a 22.4% chance that the attacker wins a 5 versus 1 battle with 4 troops remaining. Winning battles are those in which no defending troops remain, so it can be verified that for all rows:

$$\forall A, D \in \mathbb{N}^+, \sum \Omega_{(A/D),(R_A/0)} = \Omega_{(A/D),(Win)}$$

Precomputing the Blitz Probability Matrix at compile-time should theoretically improve the efficiency of the overall system. During RL training, battle outcomes can be sampled directly via table lookup rather than requiring repeated stochastic simulation at run time.

The Blitz Probability Matrix is extended to a dimension of 100. The corresponding .csv file and the augmented script used to generate it can be found in the source code for this project.

Chapter 4

Design and Implementation

4.1 Eliciting the Metadata of a Risk Map

A Risk map defines the static structural constraints under which the game is played. While gameplay dynamics such as troop movements and territory ownership evolve over time, the map itself remains fixed and determines which interactions are possible. Through the elicitation and formalisation of its defining characteristics, a general map representation is constructed that can model arbitrary Risk maps for use within a reinforcement learning environment. The classical Earth map, *EARTH-M*, is used throughout as a concrete motivating example.

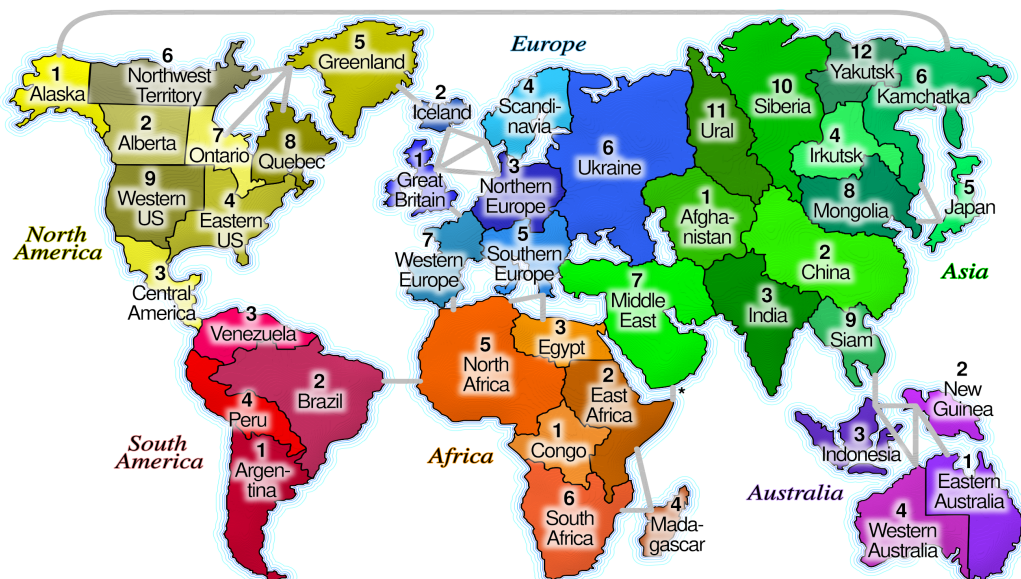


Figure 4.1: A graphical view of the classical Earth map [26].

Observe that the Earth map comprises a set of **Territories** (M_T). Each territory belongs to exactly one **Continent** (M_C), as specified by a **territory-continent mapping** (M_{TC}). Continents are significant in Risk, as players who control all territories within a continent receive additional reinforcement troops each round, as defined by a **continent-bonus mapping** (M_{CB}). Finally, territories are connected to one another via **Borders** (M_E).

Therefore, the following information is sufficient to encode any and all Risk maps:

$$M = (M_T, M_C, M_E, M_{TC}, M_{CB})$$

- M_T = Set of territories : $\{T\}$. Let T denote the data structure for a territory which for the purposes of this project will be the name of it as a string.

$$EARTH_M_T = \{\text{Brazil, China, Iceland, North Africa, ...}\}, \text{ for all 42 territories.}$$

- M_C = Set of continents : $\{C\}$. Let C denote the data structure for a continent which for the purposes of this project will be the name of it as a string.

$$EARTH_M_C = \{\text{South America, North America, Europe, Asia, Oceania, Africa}\}.$$

- M_E = Border adjacency list : $T \rightarrow \{T\}$. Since all Risk maps support bi-directional traversal, it follows that: $\forall a, b \in T, a \in M_E(b) \iff b \in M_E(a)$.

$$\begin{aligned} EARTH_M_E = \{ \\ & \text{Brazil} \mapsto \{\text{Argentina, Peru, Venezuela, North Africa}\}, \\ & \text{China} \mapsto \{\text{Siam, India, Afghanistan, Ural, Siberia, Mongolia}\}, \\ & \text{Iceland} \mapsto \{\text{Great Britain, Greenland, Scandinavia}\}, \\ & \text{North Africa} \mapsto \{\text{Congo, Brazil, Western Europe, Southern Europe, Egypt, East Africa}\}, \\ & \dots\}, \text{ for all 42 territories.} \end{aligned}$$

- M_{TC} = Territory-continent mapping : $T \rightarrow C$.

$$EARTH_M_{TC} = \{\text{Brazil} \mapsto \text{South America, China} \mapsto \text{Asia, Iceland} \mapsto \text{Europe, North Africa} \mapsto \text{Africa, ...}\}, \text{ for all 42 territories}$$

- M_{CB} = Continent-bonus mapping : $C \rightarrow \mathbb{N}$.

$$EARTH_M_{CB} = \{\text{South America} \mapsto 2, \text{ North America} \mapsto 5, \text{ Europe} \mapsto 5, \text{ Asia} \mapsto 7, \text{ Oceania} \mapsto 2, \text{ Africa} \mapsto 3\}$$

4.2 Instantiating a Map-agnostic Risk Environment

An environment ε can be depicted as follows:

$$\varepsilon = (S, A, T, R)$$

It is assumed that all games of *Risk: Global Domination* obey the same underlying ruleset, irrespective of the specific map on which they are played. This observation enables the construction of a map-agnostic Risk environment, in which the static metadata of a map is used to instantiate an environment with consistent dynamics. In particular, given a formal description of a Risk map M , an **environment-instantiation function** $\mathcal{I} : M \rightarrow \varepsilon$ is defined that generates an environment in which agents may interact, independent of the chosen map configuration.

A Note About Trading...

Manual trading of territory cards has been removed from the environment. Instead, when a player accumulates 3 territory cards, they are automatically traded in exchange for a fixed reinforcement bonus of 10 deployment troops.

This design choice reduces action space complexity and long-horizon credit assignment challenges introduced by manual trading decisions. In the standard Risk ruleset, trading involves combinatorial card selection and optional timing strategy, which substantially increases the branching factor and introduces delayed reward dependencies. Such characteristics can negatively impact policy-gradient stability and exploration efficiency in reinforcement learning.

Furthermore, because the competitive variance in Risk is primarily driven by spatial deployment and combat manoeuvres rather than the nuances of card-timing, trading strategy constitutes a relatively small portion of the overall optimal policy. Implementing an automated greedy mechanism provides a sufficient heuristic that preserves strategic pacing while focusing the agent’s learning capacity on high-impact tactical decisions. By automating the trade mechanism, the environment retains the macro-strategic pacing benefits of trading while maintaining a more tractable and RL-friendly decision space.

4.2.1 State Space (S)

Let P denote the set of all players, each player assigned a number. The state space for Risk is as follows:

$$S = (\alpha, \beta, p, \theta)$$

- α = Territory state mapping : $T \rightarrow P \times \mathbb{N}^+$, specifying the current owner and troop count of each territory. Territories must be occupied during all phases of play (i.e. $\forall t \in M_T, troops_\alpha(t) > 0$).

For instance, if $\alpha(\text{Brazil}) = (2, 7)$, then player 2 currently owns 7 troops on Brazil.

- β = Player card counts : $P \rightarrow \mathbb{N}$, specifying the amount of territory cards held by each player.
- p = Player turn : P .
- θ = Phase of play $\in \{\text{Draft}, \text{Attack}, \text{Fortify}\}$.

.

The Initial State (S_0)

$$S_0 = (\alpha_0, \{\}, 0, \text{Draft})$$

The latter 3 state variables are straightforward: Upon initialising the game it's player 0's turn entering the draft phase, and no player possesses any territory cards. Furthermore, it is assumed that the ordering of player turns is determined outside the scope of the environment itself.

The non-trivial aspect lies in determining the initial territory state mapping α_0 . To avoid overfitting to a single opening configuration, α_0 is sampled **randomly** at the start of each episode, subject to the following fairness constraints F :

- F_1 : All players must initially own the same number of troops.

$$\forall p \in P, \sum_{t \in M_T} 1[\text{owner}_{\alpha_0}(t) = p] \cdot \text{troops}_{\alpha_0}(t) = k, k \in \mathbb{N}^+$$

- F_2 : All players must approximately own the same number of territories.

$$\forall p \in P, N_p \in \{\lfloor k \rfloor, \lceil k \rceil\}, k = \frac{|M_T|}{|P|},$$

$$N_p = \text{Number of territories owned by player } p = |\{t \in M_T \mid \text{owner}_{\alpha}(t) = p\}|$$

- F_3 (optional): No two bordering territories are initially owned by the same player.

$$\forall t \in M_T, \forall u \in M_E(t), \text{owner}_{\alpha_0}(t) \neq \text{owner}_{\alpha_0}(u)$$

Unlike F_1 and F_2 , F_3 is not a canonical constraint in Risk, nor may this feature be possible to implement for certain map topologies. Nevertheless, enforcing this constraint during training can eliminate outlier episodes in which game outcomes are determined by a continent already being owned by a single player, rather than by strategic decision-making.

.

The Terminal State (S_{end})

$$S_{end} = (\alpha_{end}, \beta_{end}, p_{end}, \text{Attack})$$

The game terminates once all territories are owned by a single player. In Risk, this terminal condition is reached during the attack phase, immediately after the winning player successfully captures the final remaining territory.

$$\forall t \in M_T, \text{owner}_{\alpha_{end}}(t) = p_{end}$$

4.2.2 Action Space (A) and Transitions (T)

Deploying Troops ($\theta_t = \text{Draft}$)

The current player p_t receives U_t troops at the beginning of their turn, based on the number of territories owned (N_p), number of territory cards owned ($\beta_t(p_t)$), as well as continent bonuses:

$$U_t = \max \left(3, \left\lfloor \frac{N_p}{3} \right\rfloor \right) + \left\lfloor \frac{\beta_t(p_t)}{3} \right\rfloor \cdot 10 + \sum_{c \in M_C} 1[\forall t \in M_T, M_{TC}(t) = c \Rightarrow \text{owner}_{\alpha_t}(t) = p_t] \cdot M_{CB}(c)$$

During deployment, player p_t may distribute its incoming troops across their owned territories in any manner they choose, subject to the constraint that all U_t troops must be allocated.

$$A^{\text{deploy}} = (d(t))_{t \in M_T},$$

$$d(t) \in \mathbb{N}, \text{ owner}_{\alpha_t}(t) \neq p_t \Rightarrow d(t) = 0, \sum_{t \in M_T} d(t) = U_t$$

The resulting transition increments the troop counts of the affected territories, and subtracts any traded-in territory cards.

$$\forall t \in M_T, \text{ troops}_{\alpha_{t+1}}(t) = \text{troops}_{\alpha_t}(t) + d(t)$$

$$\beta_{t+1}(p_t) = \beta_t(p_t) \bmod 3$$

Battling Neighbouring Territories ($\theta_t = \text{Attack}$)

The current player p_t initiates a battle action by targeting a neighbouring territory not owned by them from one of their own. The attacking territory must contain at least 2 troops.

$$A^{\text{battle}} = (T_{\text{from}}, T_{\text{to}})$$

$$T_{\text{from}}, T_{\text{to}} \in M_T, T_{\text{to}} \in M_E(T_{\text{from}}), \text{ owner}_{\alpha_t}(T_{\text{from}}) = p_t, \text{ owner}_{\alpha_t}(T_{\text{to}}) \neq p_t$$

$$A = \text{troops}_{\alpha_t}(T_{\text{from}}), D = \text{troops}_{\alpha_t}(T_{\text{to}}), A \geq 2$$

After A^{battle} ends the attacker and defender are left with R_A and R_D troops respectively. The Blitz Probability Matrix Ω determines the probability distribution over (A/D) battle outcomes.

$$Pr((R_A/R_D) \mid (A/D)) = \Omega_{(A/D), (R_A/R_D)}$$

Once R_A and R_D are determined, the territory state mapping is updated.

$$troops_{\alpha_{t+1}}(T_{from}) = R_A$$

$$troops_{\alpha_{t+1}}(T_{to}) = R_D$$

Post-Capture Troop Transfer ($\theta_t = \text{Attack}$)

Following a successful battle victory, defined by the condition $R_D = 0$, the attacker must immediately perform a troop transfer into the newly captured territory.

$$A^{transfer} = k, \quad k \in \mathbb{N}^+, \quad 1 \leq k < troops_{\alpha_t}(T_{from})$$

Player p_t becomes the new owner of the captured territory and is awarded one territory card.

$$troops_{\alpha_{t+1}}(T_{from}) = troops_{\alpha_t}(T_{from}) - k, \quad troops_{\alpha_{t+1}}(T_{to}) = k$$

$$owner_{\alpha_{t+1}}(T_{to}) = p_t$$

$$\beta_{t+1}(p_t) = \beta_t(p_t) + 1$$

Upon a player elimination (i.e. $\forall t \in M_T, owner_{\alpha_{t+1}}(t) \neq owner_{\alpha_t}(T_{to})$), all territory cards are transferred to the attacker.

$$\beta_{t+1}(p_t) = \beta_t(p_t) + \beta_t(owner_{\alpha_t}(T_{to}))$$

$$\beta_{t+1}(owner_{\alpha_t}(T_{to})) = 0$$

Fortifying Existing Territories ($\theta_t = \text{Fortify}$)

During the fortify phase, the current player p_t may transfer troops between two territories they own, provided they are connected and the source territory retains at least one troop after the transfer.

$$A^{fortify} = (T_{from}, T_{to}, k),$$

$$k \in \mathbb{N}^+, T_{from}, T_{to} \in M_T, Conn(T_{from}, T_{to}), 1 \leq k < troops_{\alpha_t}(T_{from})$$

The $Conn$ function reduces the connectivity constraint to a **connected components problem** [27] on the subgraph $S = (V_p, E_p)$, where S represents the territories and edges owned by p_t . Therefore, fortification between T_{from} and T_{to} is possible if and only if they are connected in S .

$$Conn(T_{from}, T_{to}) \iff (T_{from}, T_{to}) \in CC(S)$$

In the new state, k troops are transferred from T_{from} to T_{to} . Fortification may be performed at most once per turn; afterwards, control passes to the next player into the draft phase.

$$troops_{\alpha_{t+1}}(T_{from}) = troops_{\alpha_t}(T_{from}) - k$$

$$troops_{\alpha_{t+1}}(T_{to}) = troops_{\alpha_t}(T_{to}) + k$$

$$p_{t+1} = (p_t + 1) \bmod |P|$$

$$\theta_{t+1} = \text{Draft}$$

Phase Transitions

All actions defined above are optional (except for A^{deploy} which must be executed while $U_t > 0$, and $A^{transfer}$ which must follow successful battle victories). Players also have the option to skip to the next phase of play.

$$A^{skip}$$

$$\theta_t = \text{Draft} \Rightarrow \theta_{t+1} = \text{Attack}$$

$$\theta_t = \text{Attack} \Rightarrow \theta_{t+1} = \text{Fortify}$$

$$\theta_t = \text{Fortify} \Rightarrow \theta_{t+1} = \text{Draft}, p_{t+1} = (p_t + 1) \bmod |P|$$

4.3 Declaring Heuristic Agents

The quality of heuristic agents is crucial to the success of the experimental framework in two key respects. Firstly, each heuristic agent can be viewed as an approximation of a strategy that might be adopted by a human player in Risk. Evaluating these agents as representatives of distinct tactical behaviours and analysing how their performance varies across different map topologies provides valuable insight into the effectiveness of different strategic approaches and helps inform conclusions about optimal policies.

Secondly, heuristic agents play a central role in the training process of the RL agent. Since training trajectories are generated through interactions involving these agents, their quality directly influences the learning signal available to the model. A diverse pool of heuristic agents, interchangeable between episodes, is essential to prevent the RL agent from overfitting to a single style of play and instead encourages the development of more generalisable policies.

Furthermore, the competency of these agents impacts training efficiency. For instance, consider a simulation between 2 poorly performing agents (e.g. **RandomAgent** versus **RandomAgent**): this often results in excessively long episodes with little meaningful strategic progression, yielding low-quality trajectories. In contrast, more informed heuristic agents produce shorter, more strategically rich games, enabling a higher throughput of informative training episodes per batch and ultimately improving learning efficiency.

4.3.1 Strategies

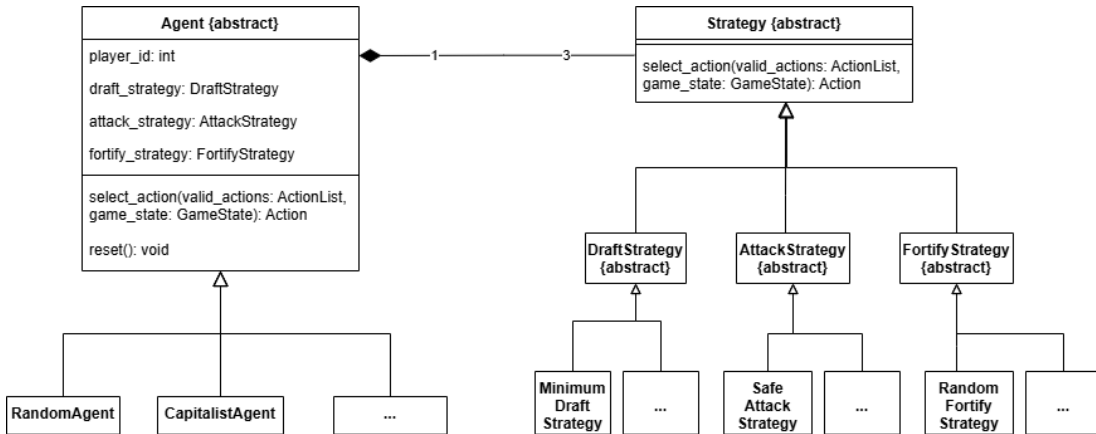


Figure 4.2: UML class diagram illustrating the heuristic agent architecture, in which each agent is composed of interchangeable strategy components governing the three phases of gameplay.

Adopting an object-oriented design based on the strategy pattern [28] (not to be confused with gameplay strategies) allows each agent to be composed of interchangeable decision-making components. By separating drafting, attacking, and fortifying into independent strategies, behaviours can be developed and tested in isolation, then easily recombined to create a range of agents. This approach keeps the system flexible, promotes code reuse, and makes it straightforward to introduce new heuristics without modifying the core agent logic.

Draft Strategies ($\theta_t = \text{Draft}$)

- `RandomDraftStrategy()`: Select a random A^{deploy} action.
- `MinimumDraftStrategy()`: Apply A^{deploy} to the territory with the fewest troops.
- `MaximumDraftStrategy(capitalCount:  $\mathbb{N}^+$ )`: Let $capitals \subseteq M_T$ denote the `capitalCount` most populous territories. Apply A^{deploy} to any random territory $t \in capitals$.

Attack Strategies ($\theta_t = \text{Attack}$)

- `WeightedRandomAttackStrategy(battleWeight:  $[0, 1]$ )`: Sample $r \sim \mathcal{U}(0, 1)$. If $r < \text{battleWeight}$, select a random A^{battle} action; otherwise apply A^{skip} . Battle victories are resolved using `transferMethod = RANDOM`.
- `SafeAttackStrategy(disparity:  $\mathbb{Z}$ , transferMethod)`: Let $a^* = (T_{from}, T_{to})$ denote the A^{battle} with the highest troop disparity. If $troops_{\alpha_t}(T_{from}) - troops_{\alpha_t}(T_{to}) \geq \text{disparity}$, apply a^* ; otherwise apply A^{skip} . Battle victories are resolved using the parsed `transferMethod`.

Fortify Strategies ($\theta_t = \text{Fortify}$)

- `RandomFortifyStrategy()`: Select a random $A^{fortify}$ or A^{skip} action.
- `MinimumFortifyStrategy()`: Select $A^{fortify}$ that splits troops from the most populous territory to the least populous territory, using `transferMethod = SPLIT`.
- `MaximumFortifyStrategy(capitalCount:  $\mathbb{N}^+$ )`: Select a random $A^{fortify}$ action that moves all troops between $T_{from} \in capitals$ and $T_{to} \notin capitals$, using `transferMethod = ALL`. This effectively shifts the capital location(s) each turn, encouraging troop redistribution and the pursuit of new attacks, thereby reducing stagnation and helping ensure games eventually conclude.

A Note About Troop Movement...

The `transferMethod` parameter is used by both battle and fortify actions to determine how many troops are to be relocated from T_{from} to T_{to} . Restricting transfer behaviour to these four predefined types reduces the effective action space from an unbounded integer range to a fixed set of four discrete options, simplifying optimisation while preserving meaningful tactical archetypes.

- **RANDOM:** Transfer a random amount of troops in the range $[1, troops_{\alpha_t}(T_{from}) - 1]$.
- **ONE:** Transfer 1 troop.
- **SPLIT:** Transfer $\max\left(1, \left\lfloor \frac{troops_{\alpha_t}(T_{from}) - troops_{\alpha_t}(T_{to})}{2} \right\rfloor\right)$ troops.
- **ALL:** Transfer $(troops_{\alpha_t}(T_{from}) - 1)$ troops.

4.3.2 Agents

RandomAgent

- **Draft Strategy:** `RandomDraftStrategy()`
- **Attack Strategy:** `WeightedRandomAttackStrategy(battleWeight=0.95)`
- **Fortify Strategy:** `RandomFortifyStrategy()`

The `RandomAgent` represents a naive, non-strategic player that selects actions without tactical reasoning. It serves as a baseline for evaluating more structured agents and highlights the performance gain achievable through principled decision-making. A high default `battleWeight` is chosen to encourage active gameplay and prevent excessive skipping, ensuring games progress toward termination.

CommunistAgent

- **Draft Strategy:** `MinimumDraftStrategy()`
- **Attack Strategy:** `SafeAttackStrategy(disparity=3, transferMethod=SPLIT)`
- **Fortify Strategy:** `MinimumFortifyStrategy()`

The **CommunistAgent** models a risk-averse player. Troops are spread to minimise disparities between territories, and attacks are initiated only when outcomes are sufficiently favourable. This results in steady, controlled expansion while maintaining a broadly balanced defensive structure.

CapitalistAgent

- **Draft Strategy:** `MaximumDraftStrategy(capitalCount=1)`
- **Attack Strategy:** `SafeAttackStrategy(disparity=5, transferMethod=ALL)`
- **Fortify Strategy:** `MaximumFortifyStrategy(capitalCount=1)`

The **CapitalistAgent** adopts a concentrated-force strategy, prioritising a small number of key strongholds rather than evenly distributing resources. Reinforcements are funnelled into a primary territory, and expansion occurs only when a substantial advantage is available. This produces burst-like aggression patterns, where periods of consolidation are followed by decisive offensives.

Agent Parameterisation and Diversity

At first glance, the use of only three heuristic agents may appear limited. However, the configurations presented above represent default parameter settings rather than fixed behavioural definitions. Each agent family is parameterised, meaning that variations in values can substantially alter gameplay dynamics. For example, a **CommunistAgent** with `disparity = 1` behaves far more aggressively than one with `disparity = 3`, while a **CapitalistAgent** configured with three capitals exhibits a noticeably different reinforcement structure compared to a single-capital variant.

To systematically introduce this variability, an **AgentSampler** is used to instantiate agents across a range of parameter values. This produces a diverse distribution of opponent behaviours without requiring entirely new strategy classes, thereby exposing the RL agent to varied play styles during training and promoting greater policy robustness.

4.4 Training the RL Agent

The reinforcement learning pipeline is implemented using the OpenAI **Gymnasium** framework, a standardised interface for developing and benchmarking reinforcement learning environments [29]. Gymnasium defines a minimal API consisting primarily of the `reset()` and `step()` functions, enabling learning algorithms to interact with arbitrary environments in a consistent and modular manner. By adhering to this interface, the project benefits from compatibility with widely used reinforcement learning libraries, including the aforementioned **MaskablePPO**, while maintaining a clear separation between environment dynamics and learning logic.

To integrate the native **RiskEnv** game engine with the Gymnasium framework, a wrapper class, **RiskGymEnv**, is introduced. Its primary role is to translate the internal game representation into structured **observations** compatible with the Gym API and neural network inputs. It also decodes policy outputs into valid game actions, applies action masking, computes rewards, and advances the environment until control returns to the RL agent. This abstraction cleanly separates learning logic from core game mechanics.

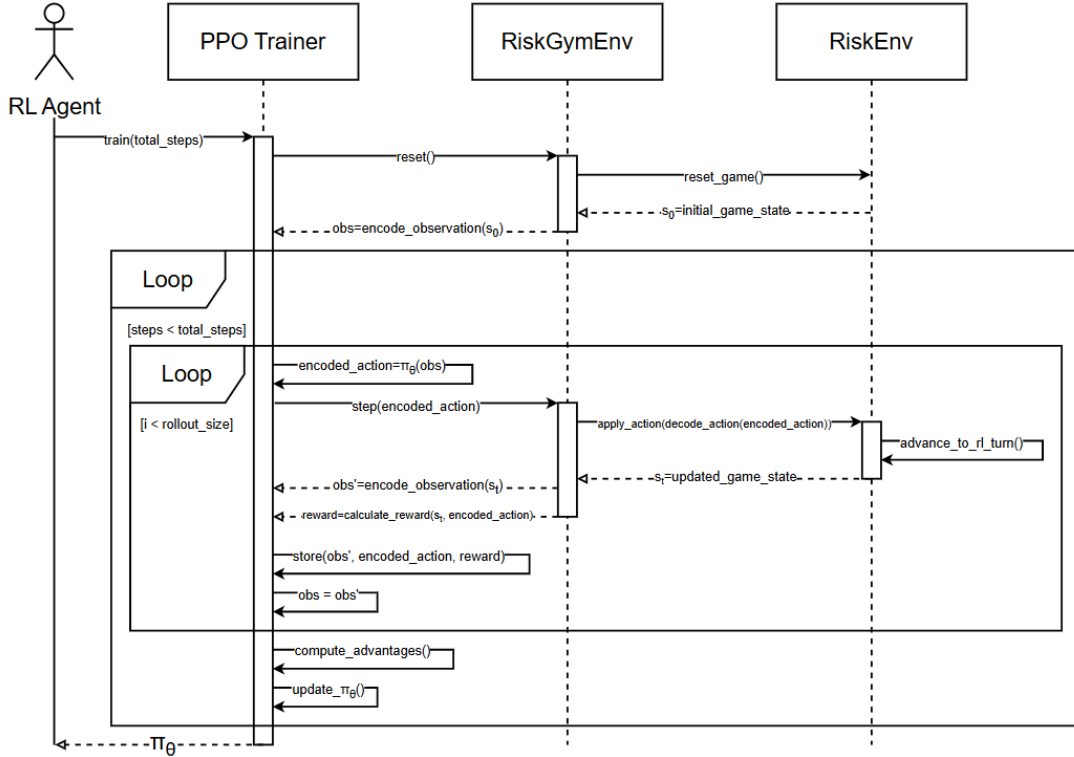


Figure 4.3: UML sequence diagram illustrating the interactions between the **PPO Trainer**, **RiskGymEnv**, and **RiskEnv** during the training phase. For conciseness, the diagram omits the environment reset procedure typically triggered upon reaching a terminal state.

4.4.1 Implementing `calculate_reward()`

The **reward function** ($R(s, a) : S \times A \rightarrow \mathbb{R}$) is designed to evaluate strategic progress at the granularity of complete turns taken by the RL agent, $\mathcal{R}$, rather than individual atomic actions. In a single turn, the agent typically performs a sequence of 15-20 actions. Assigning reward only at the end of the game results in an extremely sparse learning signal, leading to slow credit assignment across hundreds of actions. Conversely, assigning reward after every atomic action introduces high variance and risks encouraging short-sighted, locally optimal behaviour.

The following approach represents a principled compromise between these extremes. By evaluating the state transition from the beginning of the agent's turn to the point at which control returns after all opponents have acted, the reward signal remains sufficiently dense to enable efficient learning, while still reflecting meaningful strategic outcomes. In this sense, the design balances the bias-variance trade-off inherent in reward shaping and temporal credit assignment.

Formally, let $S_{\mathcal{R}_0} \subset S$ denote the set of states in which the RL agent is about to begin its turn, and let $S_{\text{end}} \subset S$ denote the set of terminal states. The reward is defined as:

$$R(s, a) = \begin{cases} +1 & \text{if } s \in S_{\text{end}} \text{ and } p_{\text{end}} = \mathcal{R}, \\ -1 & \text{if } s \in S_{\text{end}} \text{ and } p_{\text{end}} \neq \mathcal{R}, \\ \lambda_1 \Delta T R_T + \lambda_2 \Delta T E_T + \lambda_3 \Delta C_T & \text{if } s \in S_{\mathcal{R}_0}, \\ 0 & \text{otherwise.} \end{cases}$$

The first two cases correspond to terminal states. If the episode has ended and the RL agent $\mathcal{R}$ is the winner ($p_{\text{end}} = \mathcal{R}$), a reward of +1 is assigned; otherwise a reward of -1 is given. This ensures that long-term strategic success remains the dominant optimisation objective.

The third case applies at the beginning of each RL turn. Let $s_T \in S_{\mathcal{R}_0}$ denote the state at the beginning of the agent's turn and $s_{T+1} \in S_{\mathcal{R}_0}$ the next such state (i.e. after the agent has acted and all opponents have completed their turns). The intermediate reward is computed using strategic deltas:

$$\Delta T R_T = \frac{\sum_{t \in M_T} 1[\text{owner}_{\alpha_{T+1}}(t) = \mathcal{R}] \cdot \text{troops}_{\alpha_{T+1}}(t)}{\sum_{t \in M_T} \text{troops}_{\alpha_{T+1}}(t)} - \frac{\sum_{t \in M_T} 1[\text{owner}_{\alpha_T}(t) = \mathcal{R}] \cdot \text{troops}_{\alpha_T}(t)}{\sum_{t \in M_T} \text{troops}_{\alpha_T}(t)}$$

$$\Delta TE_T = \frac{\sum_{t \in M_T} 1[\text{owner}_{\alpha_{T+1}}(t) = \mathcal{R}] - 1[\text{owner}_{\alpha_T}(t) = \mathcal{R}]}{|M_T|}$$

$$\Delta C_T = \frac{\sum_{c \in M_C} (1[\forall t \in M_T, M_{TC}(t) = c \Rightarrow \text{owner}_{\alpha_{T+1}}(t) = \mathcal{R}] - 1[\forall t \in M_T, M_{TC}(t) = c \Rightarrow \text{owner}_{\alpha_T}(t) = \mathcal{R}]) \cdot M_{CB}(c)}{\sum_{c \in M_C} M_{CB}(c)}$$

Here, the troop delta ΔTR_T captures relative military strength, preventing bias toward purely territorial expansion strategies. The territory delta ΔTE_T measures board control, while the continent delta ΔC_T reflects acquisition of reinforcement bonuses and long-term strategic advantage.

The weighting coefficients satisfy $\lambda_3 > \lambda_2 > \lambda_1 > 0$, ensuring that continent acquisition is valued most highly, followed by territorial expansion, and finally relative troop share. This ordering reflects the strategic hierarchy of the game while maintaining a simple linear reward structure.

The final case assigns zero reward to all intermediate atomic states within a turn, preserving the turn-level credit assignment scheme described above.

4.4.2 Gym-like Optimisations

Action Atomicity

Recall that the output layer of the policy network represents a flattened enumeration of all available actions. For efficient learning and improved generalisation, it is desirable for the action space A to be as small as possible while preserving the semantic meaning of the underlying game mechanics. One effective strategy is to decompose combinational actions into smaller, sequential atomic operations, thereby reducing per-step decision complexity without sacrificing expressiveness.

Consider A^{deploy} . In a fully combinational formulation, the agent selects a complete allocation vector distributing U_t troops across N_p player-owned territories in a single time step. The number of such allocations is given by $\binom{U_t + N_p - 1}{N_p - 1}$. In a typical mid-game scenario one might have $N_p \approx 14$ and $U_t \approx 10$, yielding $\binom{23}{13} = 1,144,066$ possible deployment actions at a single decision point. A better alternative is to deploy one troop per action, shrinking the action

space to N_p . The full deployment plan is then realised as a short sequence of atomic actions, reducing combinational complexity from exponential in U_t to linear in N_p per step.

A similar argument applies to $A^{fortify}$. If the source territory, destination territory, and transfer quantity are chosen jointly, the action space scales as $((N_p)^2 \cdot troops_{\alpha_t}(T_{from}))$. Decomposing this into sequential actions $A^{fortifyFrom}$, $A^{fortifyTo}$, and $A^{fortifyAmount}$ reduces the per-step complexity to $(N_p + N_p + troops_{\alpha_t}(T_{from}))$, which is linear rather than multiplicative.

Observation Feature Engineering

While the raw **GameState** provides a complete description of the environment, not every data point is useful for policy optimisation. The observation presented to the neural networks therefore utilises **feature selection**. Only features directly relevant to decision making are retained (territory ownership, troop counts, current phase etc.), whereas information such as absolute player indices or opponent-specific bookkeeping variables are treated as noise and are thus removed. This reduces input dimensionality, improving sample efficiency by ensuring the policy and value networks focus only on strategically meaningful signals.

Categorical variables, such as the current phase, are represented using **one-hot encoding**. One-hot encodings prevent the network from inferring artificial ordinal relationships between discrete categories (e.g., treating phase index 2 as “larger” than phase index 1), and instead enforce a sparse, semantically neutral representation. This typically improves gradient behaviour and stabilises learning, particularly in policy-gradient methods such as PPO.

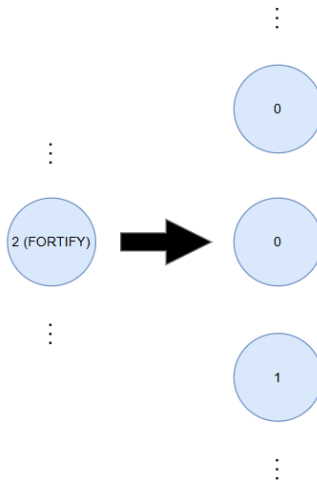


Figure 4.4: One-hot encoding applied to the current phase feature vector.

All scalar quantities are **normalised** to the range $[0,1]$. Troop counts are scaled relative to the total number of troops currently on the board, and card counts are normalised by their maximum possible value. Normalisation ensures consistent feature magnitudes across inputs, preventing large-valued features from dominating gradient updates. This improves optimisation stability, accelerates convergence, and allows the policy and value networks to learn scale-invariant representations of strategic strength.

Invalid Action Masking

To navigate the expansive action space of Risk, the `RiskGymEnv` implements an `action_masks()` method that returns a binary vector of valid moves for any given state. This method provides the necessary interface for the `MaskablePPO` algorithm to enforce game rules during the sampling process. By masking out illegal moves, such as attacking a non-adjacent territory, the agent is prevented from ever selecting an invalid action. This optimisation constrains the policy to valid trajectories, which significantly accelerates convergence by ensuring the agent does not waste valuable training samples on prohibited moves.

4.4.3 PPO Inference

As a concrete implementation of the `Agent` interface, the RL `Agent` is required to provide a `select_action()` method to facilitate gameplay. Assuming the policy network π_θ has been sufficiently trained, action selection is computationally efficient, requiring only a single forward pass through the architecture, supplemented by the encoding of the raw game state into a network-ready observation and the subsequent decoding of the predicted action into an executable game command.

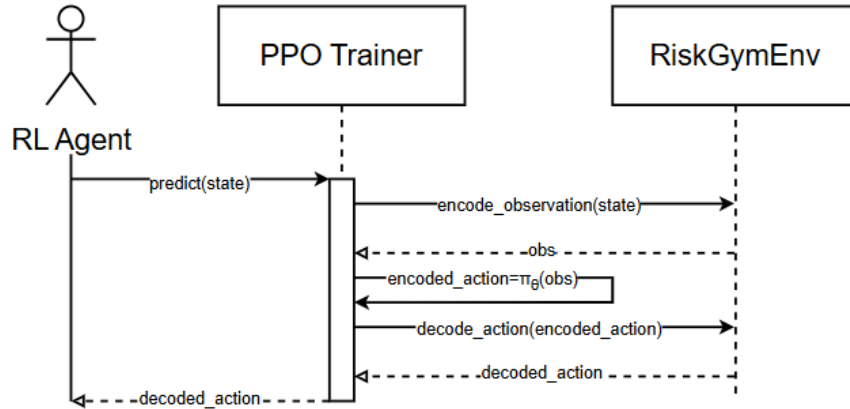


Figure 4.5: UML sequence diagram illustrating the interactions between the `PPO Trainer` and `RiskGymEnv` during the inference phase.

Chapter 5

Experiments

5.1 Experiment 1: Effect of Map Density on Concentrated vs Distributed Strategies

5.1.1 Brief

Hypothesis: Concentrated strategies outperform distributed strategies on dense maps, but underperform on sparse maps.

For any given map M , its density M_ρ is defined as the ratio of the number of borders with respect to the maximum possible borders [30]:

$$M_\rho = \frac{2|M_E|}{|M_T|(|M_T| - 1)}$$

The underlying intuition is that high-density maps ($M_\rho \rightarrow 1$) increase the number of exposed borders, making it difficult to defend large regions. In such environments, concentrating forces into a small, well-fortified core is expected to be advantageous. Conversely, sparse maps ($M_\rho \rightarrow 0$) typically feature fewer borders per territory, making larger regions easier to defend. This favours strategies that expand gradually and maintain broader territorial control. Additionally, greater opportunities exist to exploit the deployment bonus U_t , which scales with the map size $|M_T|$.

Independent Variables

- Map density (M_ρ): Increases from $\frac{2}{M_T} \rightarrow 1$ at an interval of 0.01.

Dependent Variables

- Win rate per agent.
- Average finish position per agent.

Controlled Variables

- Map size $|M_T|$: All games will be played on a map that has 16 territories, ensuring that map density will only be affected by adding/removing borders.
- Number of continents: The entire map is a shared continent; continent bonuses are a non-factor in this experiment.
- Number of episodes per simulation: 1000 games will be played per map before the win rates and average finish positions are calculated.
- Agent composition: 4 players, including a **CapitalAgent** and **CommunistAgent** to represent concentrated and distributed playstyles respectively. 2 additional **RandomAgents** are also included to introduce stochasticity and better reflect a varied gameplay environment.
- Turn ordering: Varies between episodes to ensure results are not skewed by ordering effects.

Methodology

A map in Risk must be connected (i.e. all territories are reachable from each other) in order to be playable. Therefore, a minimum of $(|M_T| - 1)$ borders are required, resulting in a density lower bound of $\frac{2}{M_T}$.

To facilitate the experiment, a **KCliqueGenerator** ($\omega(k, M_\rho) : \mathbb{N}^+ \times \mathbb{R}^+ \rightarrow M$) takes in a clique size k and density $M_\rho \geq \frac{2}{k}$, and produces a single-continental, connected map containing k territories, with density M_ρ as follows:

1. Let $M_T = \{t_1, \dots, t_k\}$. Define the set of all borders $E^{\max} = \{(t_i, t_j) \in M_T \times M_T \mid i < j\}$, where $|E^{\max}| = \frac{k(k-1)}{2}$.
2. Initialise $E = \{(t_i, t_{i+1}) \mid 1 \leq i < k\}$ to form a spanning chain, ensuring minimal connectivity between all territories.
3. Let $E^{\text{rem}} = E^{\max} \setminus E$. Randomly permute its elements, and iteratively add edges to E until $|E|$ reaches the target size of $\lfloor M_\rho \cdot |E^{\max}| \rfloor$.
4. Construct the adjacency list M_E such that $\forall (t_i, t_j) \in E, t_j \in M_E(t_i)$ and $t_i \in M_E(t_j)$.
5. Finally, $M_C = \{C_{\text{Global}}\}$, $M_{TC}(\forall t \in M_T) = C_{\text{Global}}$, and $M_{CB}(C_{\text{Global}}) = 0$.

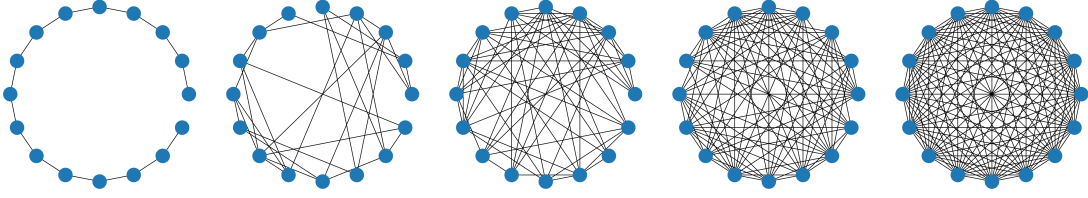
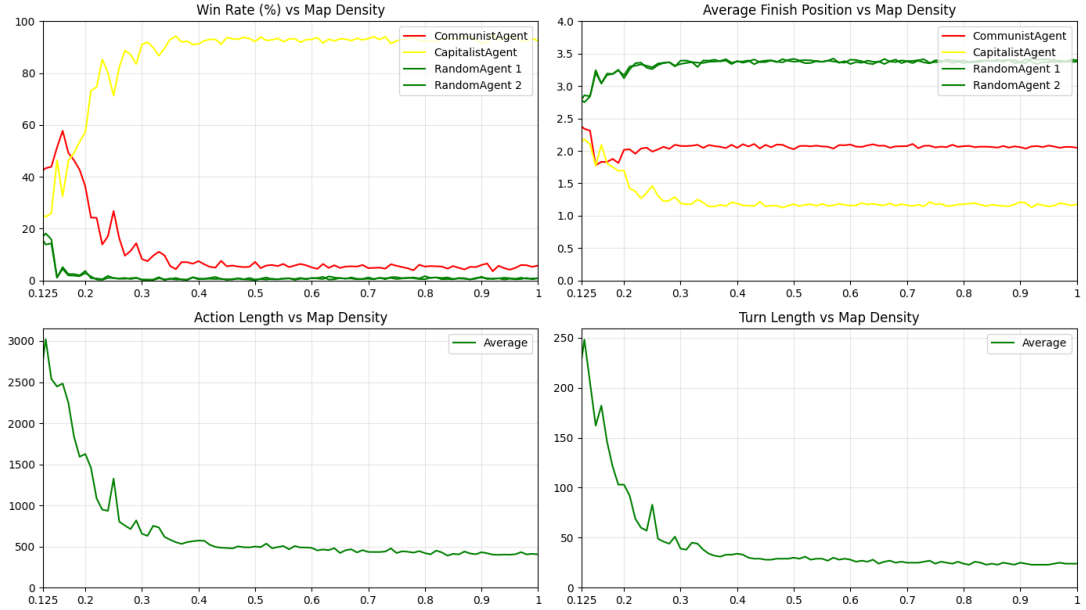


Figure 5.1: Risk maps created by $\text{KCliqueGenerator}(k = 16, M_\rho = [0.125, 0.25, 0.5, 0.75, 1])$.

5.1.2 Data Collection

A summary of results for experiment 1 is as follows:



5.1.3 Result Evaluation

Strategy Performance and the Density Crossover

As hypothesised, the experimental results demonstrate a clear phase transition in strategic viability as a function of map density. The **CommunistAgent** excels in sparse environments, while the **CapitalistAgent** achieves dominance as density increases. Notably, the crossover point occurs remarkably early, at approximately $M_\rho \approx 0.17$.

This rapid shift is likely attributed to the relatively small map size ($|M_T| = 16$). In such a constrained graph, the addition of even a few edges significantly reduces the diameter, allowing the **CapitalistAgent**’s single massive stack to project power across the entire map effectively. It is expected that for larger networks, this transition would shift further to the right, as greater geographic distance would preserve the viability of distributed territorial defence for longer.

Furthermore, comparing win rates and average finishing positions reveals a survivalist advantage for the concentrated strategy. Even where **CommunistAgent** holds a higher win probability, **CapitalistAgent** maintains a superior average finish by avoiding early elimination. While concentration struggles on bottlenecked maps, it offers stability; conversely, the distributed strategy is volatile, either winning or collapsing rapidly due to defending multiple thin fronts.

Logarithmic Trends

Another key observation is the logarithmic rate of change in the graphs, plateauing as M_ρ approaches 1.0. This behaviour reflects diminishing returns in topological redundancy. In a sparse graph, a single edge can fundamentally bridge two distant regions or remove a critical bottleneck. However, as the map approaches a complete graph, additional connectivity provides no marginal impact on the game dynamics.

Geographic Luck

The performance of the **RandomAgent** provides unexpected insight into the relationship between map sparsity and stochasticity. Although lacking any strategic heuristic, the **RandomAgent** maintains a non-zero win rate in the sparsest environments. This, coupled with the fact that even the dominant **CommunistAgent** fails to exceed a 60% win rate at minimum density, suggests that sparse topologies introduce a high degree of geographic luck.

Temporal Dynamics and the Bottleneck Effect

The data regarding action and turn lengths reveal a significant "bottleneck effect" in sparse maps. Games played at the minimum density are exponentially longer because the requirement for total world domination is hampered by restrictive movement paths. In these environments, the **CommunistAgent** can stall the **CapitalistAgent** by forcing it to navigate through narrow chains of territories.

This delay can be explained by the **CapitalistAgent**'s movement logic; because it moves its "capital" to a random neighbour during fortification, it often spends many turns wandering into "dead-end" branches of a sparse map rather than hunting the final enemy. As M_ρ increases, the number of neighbours per territory grows, rendering these defensive stalemates untenable and ensuring that even "random" movement eventually brings the concentrated stack into contact with the opposition.

5.1.4 Threats to Validity

Internal Validity

Internal validity refers to the extent to which an experiment can conclude that the independent variable is the true cause of the effect [31]. One primary threat is the inherent stochasticity of the game environment. Since combat outcomes and territory distributions are non-deterministic processes, a single trial might reflect a "lucky" streak rather than a superior strategy. To mitigate this, results are averaged over a large sample size of simulations to ensure that the observed trends in win rates and finish positions are statistically significant.

Furthermore, the specific implementation of the agent heuristics represent a potential confounding variable. The **CapitalistAgent** and **CommunistAgent** are fixed implementations and therefore, the results reflect the performance of these specific algorithms rather than the abstract concepts of concentration or distribution in their entirety. It is possible that a more sophisticated version of a distributed strategy could overcome the weaknesses identified here, meaning the results are tied closely to the heuristic logic provided.

External Validity

External validity refers to the extent to which the results of an experiment can be generalised to the real world [31]. A significant consideration here is the behaviour of the agents compared to human players. While the agents follow logical heuristics, they do not model the psychological factors, such as negotiation, revenge, or erratic decision-making that characterise human Risk play.

It should be noted that some of the maps generated in this experiment are unlikely to be fully representative of canonical Risk boards. Firstly, the map densities used in several iterations are significantly higher than those found in standard maps. For context, the classic Earth map in Risk has a density $EARTH_M\rho = \frac{2(83)}{42(41)} \approx 0.096$, whereas some experimental runs used much denser configurations. Furthermore, traditional Risk boards form a planar graph, meaning most territories border only 3 or 4 neighbours on average, a structural constraint characteristic of planar graphs (e.g. as studied in results such as the Four Colour Theorem [32]). By contrast, the generated maps in this experiment are not restricted to planar embeddings, allowing substantially higher degrees. Although such maps are indeed rare in competitive play, their inclusion in the experiment was ultimately worthwhile in exposing the theoretical limits of each strategy’s defensive logic.

5.2 Experiment 2: Effect of Map Size on First-Mover Advantage in Two-Player Risk

5.2.1 Brief

Hypothesis: In two-player Risk games, the player who takes the first turn possesses an inherent advantage, irrespective of map size.

Balancing turn order is a longstanding challenge in competitive board game design. Many turn-based games must carefully consider whether the player who acts first gains an inherent structural advantage, and if so, whether compensatory mechanisms are required to preserve fairness. The same principle applies to Risk, where turn order may influence early expansion and long-term game state development.

In this experiment, the agent assigned the first move is labelled **Player 1**, and the agent assigned the second move is labelled **Player 2**. The central premise is that Player 1 initiates offensive actions before Player 2 has the opportunity to respond. Since reinforcement troops are awarded proportionally to the number of territories owned, early territorial gains may snowball over subsequent turns. Furthermore, because territory cards are acquired following successful attacks, Player 1 may begin accumulating strategic resources sooner. Consequently, Player 2 may consistently operate in a reactive state, potentially lagging behind in development throughout the game.

Independent Variables

- Map size ($|M_T|$): Increases from 20 $\rightarrow$ 100 at an interval of 1. Since the map density is fixed, this causes the borders of the graph to also scale. Again, the `KCliqueGenerator` is utilised for making the different map topologies.

Dependent Variables

- Win rate for **Player 1**.

Controlled Variables

- Map density (M_ρ): Fixed at 0.1 for all simulations. This value reflects a moderate and realistic connectivity level comparable to standard Risk-style maps.

- Number of continents: The entire map is a shared continent; continent bonuses are a non-factor in this experiment.
- Number of episodes per simulation: 1000 games will be played per map before **Player 1**'s win rate is calculated.
- Agent composition: 2 players, both **RandomAgents**. Crucially, both agents follow identical stochastic policies, ensuring that any difference in performance arises solely from turn order rather than behavioural asymmetry.
- Turn ordering: **Player 1** always acts first, **Player 2** always acts second.

Methodology

Each game on a given map M is treated as an independent Bernoulli trial [33], where a success corresponds to a win by **Player 1**, and a failure corresponds to a win by **Player 2**. Let X denote the number of successes observed over N trials. Under the null hypothesis that both agents have an equal probability of winning:

$$X \sim \text{Binomial}(N, 0.5)$$

$$H_0 : P(\text{Player 1 wins on } M) = 0.5$$

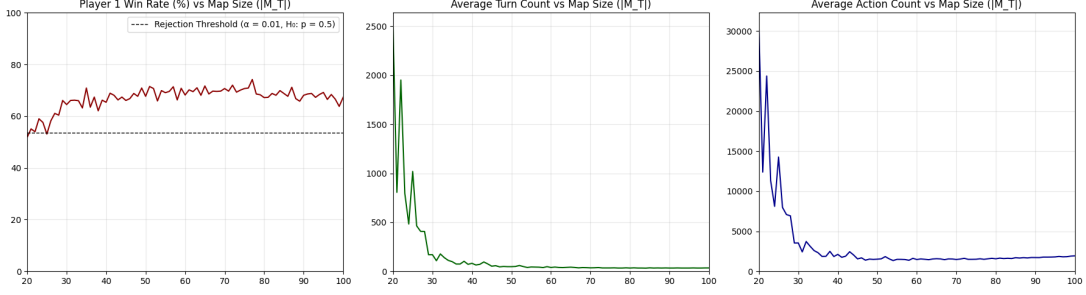
The alternative hypothesis is defined as:

$$H_1 : P(\text{Player 1 wins on } M) > 0.5$$

A one-tailed hypothesis test is conducted with significance level $\alpha = 0.01$. The null hypothesis is rejected if the probability of observing a value of X as extreme or more extreme than the observed outcome under the binomial model is less than α . Since $N = 1000$, and using the normal approximation to the binomial distribution, this corresponds approximately to observing at least 537 wins by **Player 1** (i.e. a win rate $\geq 53.7\%$). Rejection of the null hypothesis indicates a statistically significant first-mover advantage on map M , providing evidence of ordering effects.

5.2.2 Data Collection

A summary of results for experiment 2 is as follows:



5.2.3 Result Evaluation

First-Mover Advantage Across Map Sizes

The experimental data confirms a persistent first-mover advantage across the tested range, with **Player 1** maintaining a win rate above the statistical rejection threshold for the majority of map sizes. This provides strong evidence that turn order introduces a structural imbalance in two-player Risk games.

For smaller maps ($|M_T| < 30$), however, the win rates approach parity and occasionally fall below the rejection threshold. One plausible explanation is that reinforcement scaling does not amplify sufficiently in such environments. On smaller maps, the effective deployment bonus is limited, restricting the early advantage that **Player 1** can establish. As a result, **Player 2** has a greater opportunity to counter-attack before control is consolidated. This closer balance may also lead to longer and more volatile games between the **RandomAgents**, as neither agent can secure decisive dominance in the early stages.

The Snowball Threshold and the Performance Ceiling

As map size increases beyond 35 territories, the average turn and action counts drop sharply and stabilise, while the win rate for **Player 1** rises to a plateau in the range of 65%-70%. This indicates a "Snowball Threshold" where the map becomes large enough for the first agent to secure a critical mass of territories in the opening rounds.

Once this threshold is reached, the reinforcement advantage becomes very difficult for a reactive agent to overcome. However, the plateau itself indicates that the inherent weakness of

the **RandomAgent** policy places an upper limit on the observed first-mover advantage. Even with a strong territorial lead, the stochastic behaviour of **Player 1** means that **Player 2** still retains an approximate 30% chance of victory. These wins likely occur when **Player 1** fails to consolidate forces effectively or attempts unfavourable attacks. With a stronger and more consistent policy, this win rate could potentially approach 100%.

5.2.4 Threats to Validity

Internal Validity

The primary threat to internal validity arises from the use of **RandomAgent** policies. These agents do not represent optimal or strategically consistent play, and theoretically the experiment would be stronger if the optimal policy were to be used instead. However, training a dedicated model per map configuration would introduce substantial computational overhead and was beyond the scope of this study. Importantly, both players utilise the identical policy, ensuring that any observed performance differences are attributable solely to turn order.

It is also important to emphasise that the purpose of the average turn and action count metrics was not to demonstrate that larger maps inherently produce shorter games, but rather to supplement the win-rate analysis by providing insight into the structural dynamics underlying the snowball effect.

External Validity

The maps produced in the experiment were generated using the **KCliqueGenerator**. Although map sizes and connectivity densities were deliberately selected to approximate realistic Risk configurations, the generated maps do not perfectly replicate the geographic and structural properties of official boards. In particular, continent bonuses were not modelled, which may understate the true magnitude of ordering effects, as first players in standard gameplay can exploit early continent control to obtain additional reinforcement bonuses in order to further amplify their initial advantage.

Furthermore, the study considers only two-player Risk games, whereas the traditional game is typically played with three or more participants. Player ordering effects in multi-agent settings may interact differently due to shifting alliances, elimination dynamics, and coalition behaviour.

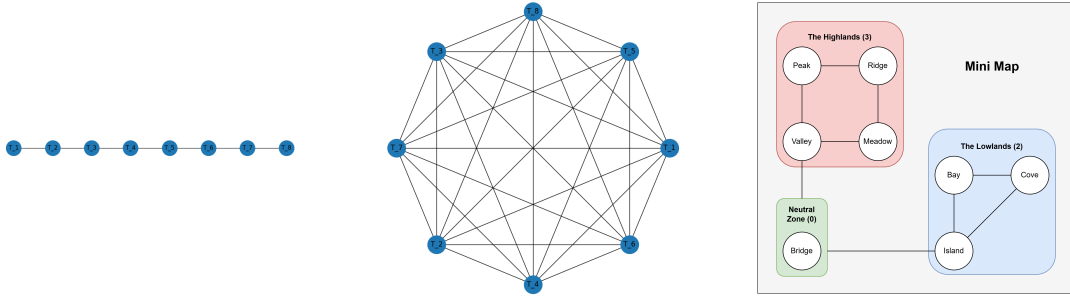
5.3 Experiment 3: Qualitative Behavioural Study of the RL Agent

5.3.1 Brief

Unlike the preceding experiments, which adopted a more focused and hypothesis-driven approach, this study takes a deliberately informal and exploratory perspective. Rather than evaluating a specific performance objective, it investigates the **RL Agent**'s behaviour across different map topologies of equal size, aiming to uncover interpretative insights into its learned policy and to highlight broader structural takeaways relating to Risk-style map design.

Independent Variables

- Map (M). The following maps are used for training and evaluation of the **RL Agent**:



The $8MIN_M = \text{KCliqueGenerator}(k = 8, M_\rho = 0.25)$ map aims to assess how the **RL Agent** behaves under restricted mobility and limited offensive opportunity, requiring the agent to navigate bottleneck-heavy pathing and maintain stable defensive frontiers.

Conversely, the $8MAX_M = \text{KCliqueGenerator}(k = 8, M_\rho = 1.0)$ map aims to assess how the **RL Agent** adapts its engagement behaviour in an environment where the total lack of structural bottlenecks forces the agent to operate under constant, multi-directional threat across all owned territories.

Finally, the $MINI_M$ map is a manually defined topology designed to introduce strategic complexity that the generated clique maps lack. Crucially, this map incorporates a continent-bonus system, rewarding players with additional deployment troops for securing entire sub-regions (e.g. 3 bonus troops for controlling "The Highlands"). This addition serves to evaluate the extent to which the **RL Agent** prioritises high-value geographical objectives over brute-force territorial acquisition.

Dependent Variables

- **RL Agent** win rate.
- **RL Agent** deploy distribution: A heat-map indicating the territories on which the agent most frequently performs A^{deploy} .
- **RL Agent** battle distribution: A heat-map indicating the territories on which the **RL Agent** most frequently performs A^{battle} , considering T_{to} only.
- **RL Agent** temporal battle statistics: Measure of combat engagement over time, including battle frequency and average battle disparity ($troops_{\alpha_t}(T_{from}) - troops_{\alpha_t}(T_{to})$) per turn-step.

Controlled Variables

- Map size $|M_T|$: All maps contain exactly 8 territories, ensuring structural comparison is not confounded by scale.
- Number of episodes per simulation: 10000 games are played on each map.
- Agent composition: Two-player setting consisting of the **RL Agent** and a randomly selected baseline agent from the heuristic pool.
- Turn ordering: Varies between episodes to mitigate first-move advantage.

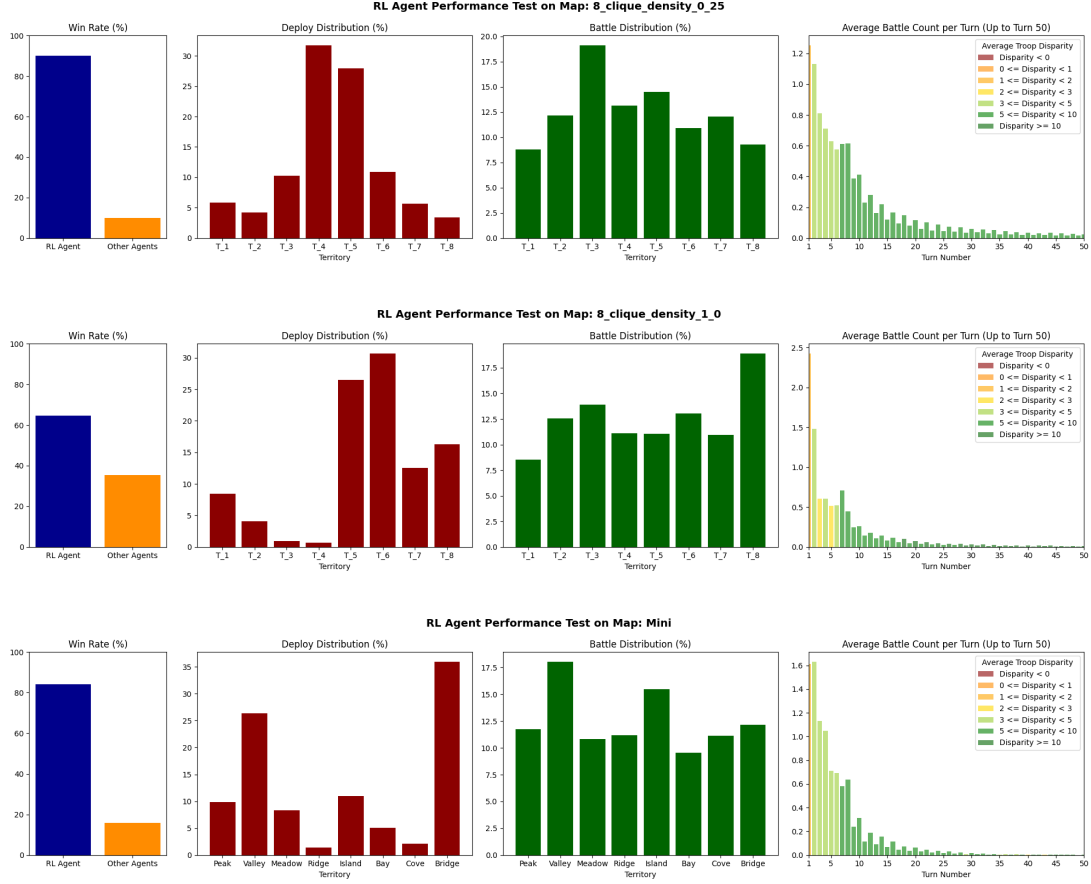
Methodology

For each map configuration, the **RL Agent** was trained independently within its respective environment. Multiple model iterations were developed over time, with hyperparameters of the **PP0 Trainer** adjusted between training runs in order to identify a stable and high-performing policy for each topology.

The model selected for evaluation on each map was the version achieving the highest empirical win rate during testing. As the focus of this experiment is behavioural analysis rather than controlled comparison of training dynamics, variations in hyperparameters across maps are not treated as a controlled variable.

5.3.2 Data Collection

A summary of results for experiment 3 is as follows:



5.3.3 Result Evaluation

Win Rate Volatility

While the RL Agent achieved consistently strong performance on both $8MIN_M$ and $MINI_M$ maps, recording win rates exceeding 80% across 10,000 evaluation episodes, it notably underperformed on $8MAX_M$. Despite multiple training iterations and adjustments to reward weighting and PPO Trainer hyperparameters, the best observed performance on this map plateaued at approximately 63%.

It is worth noting that $MINI_M_\rho = \frac{2(9)}{8(7)} \approx 0.321$, aligning it with a similar density to $8MIN_M$. Thus, the argument can be made that the observed performance dip may be attributed not only to potential sub-optimal training, but also to the inherent game balance imposed by the $8MAX_M$ topology. For example, the constant multi-directional threats induced by the high border count in $8MAX_M$ lead to a greater number of combat engagements

per turn, as reflected in the elevated average battle count per turn in the results. Given that combat resolution is inherently stochastic, an increased frequency of battles introduces additional variance into game outcomes. Consequently, the environment becomes less strategically deterministic, reducing the extent to which the RL **Agent** can reliably convert positional advantages into consistent victories.

Spatial Distribution Patterns

The deployment distribution for *8MIN_M* resembles a bell-like curve, with a concentration of reinforcements on the centrally positioned territories T_4 and T_5 . In a sparsely connected environment, such territories function as structural intermediaries linking otherwise separated regions. The agent’s preference for reinforcing these nodes suggests an emergent prioritisation of positional leverage, favouring territories that maximise both defensive coverage and future expansion potential.

For *MINI_M*, reinforcement behaviour followed a different pattern. The most frequently reinforced territories: **Valley**, **Island**, and **Bridge**, are all located along continent borders or serve as gateways between regions. Particularly, the **Bridge** received over 35% of total deployments; when analysed alongside the high battle frequencies in the **Valley** and **Island**, it is evident that the agent’s policy prioritises the disruption of enemy continent bonuses, leveraging timed attacks from the **Bridge**. More broadly, this aligns with established Risk dynamics, where decisive actions at continent-border territories often exert influence over the outcome of the game.

Temporal Aggression and Risk Management

The temporal battle graph shows that, for all maps, the RL **Agent** is most active during the first five turns of the game. During this early phase, it engages in frequent and comparatively higher-risk attacks to secure a territorial lead and leverage the "snowball effect" identified in Experiment 2. Taking land at the start of the game increases the agent’s troop income, establishing a resource advantage before the board becomes too crowded or stable. As the game progresses, battle frequency drops significantly. This occurs because the cumulative reinforcements from earlier turns lead to higher troop counts across the board, making a failed attack much more costly to the agent’s overall standing. Consequently, in the later stages of the game, the agent becomes highly risk-averse, only initiating combat when it possesses a massive troop disparity to guarantee a successful outcome.

5.3.4 Threats to Validity

Internal Validity

One limitation arises from the aggregation of results across multiple heuristic baseline agents. For clarity and ease of interpretation, outcomes were reported in grouped form rather than broken down by individual opponent type. However, this aggregation means the reported performance reflects results against a pooled set of strategies rather than disaggregated matchups. As such, it is possible that the **RL Agent** performs disproportionately poorly against a specific heuristic strategy, with this weakness being obscured when averaged across all opponents. Without finer-grained matchup analysis, conclusions regarding robustness across opponent styles remain partially unresolved.

A second threat concerns the optimality of the learned policy itself. Although multiple training runs and hyperparameter configurations were explored, the **RL Agent** cannot be assumed to have converged to a globally optimal policy. In particular, the reduced win rate observed on *8MAX_M* cannot be attributed solely to map topology, as further improvements in reward shaping, exploration scheduling, or **PPO Trainer** tuning may yield stronger performance. Consequently, some portion of the observed performance volatility may reflect training limitations rather than purely structural effects.

External Validity

The experimental maps, while controlled for structural comparison, are substantially simpler than full-scale Risk maps such as the traditional world map. With only eight territories, the strategic depth, continent interactions, and long-term planning requirements are inherently reduced. Therefore, the behavioural patterns observed here may not directly extrapolate to larger, more complex environments such as *EARTH_M*.

Additionally, the environment configuration deviates from official Risk rule sets. Certain mechanics (e.g. card trading systems and escalating reinforcement bonuses) were omitted for simplicity and training stability. Because the **RL Agent** learned within these constrained rules, its behaviour reflects environment-specific dynamics. Performance and strategic tendencies may differ if trained under the complete official rule framework.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project developed a structured and extensible experimental platform for analysing strategic behaviour in Risk as a function of map topology. By abstracting and instantiating structural properties such as connectivity, density, and bottleneck formation, it became clear that even subtle topological variations can significantly alter strategic incentives, aggression dynamics, and overall game balance. Map structure does not merely shape the difficulty of play, but actively influences which strategies become viable or dominant. Beyond the specific experiments explored in this study, the resulting system provides a modular and reusable interface for introducing new map configurations and agent types, facilitating the design and execution of further experimental evaluations.

In parallel, a generalised Proximal Policy Optimisation (PPO) training pipeline was implemented to train reinforcement learning agents across arbitrary map configurations without requiring modification to the learning algorithm. This demonstrated that stable and interpretable strategic patterns can emerge within a stochastic, long-horizon domain such as Risk. The learned policies adapted meaningfully to structural differences between maps, reinforcing the central finding that map design and agent behaviour are strongly interrelated.

6.2 Future Work

6.2.1 Reinforcement Learning Enhancements

A primary limitation of the current reinforcement learning experiments was computational capacity. All training was conducted locally without access to dedicated GPU acceleration, significantly constraining the scale of experiments that could be performed. As a result, training was largely restricted to smaller map instances with fewer players, where episode length and state-space complexity remained manageable. Given that GPU acceleration plays a central role in modern reinforcement learning workflows, access to more powerful hardware would enable substantially longer training runs and experimentation on larger, more complex configurations, including the classical Risk map. This would allow for deeper convergence analysis and more robust evaluation of strategic stability.

Beyond hardware improvements, algorithmic refinements present further opportunities for performance gains. Structured self-play, in which the agent iteratively trains against previous versions of itself as well as fixed heuristic opponents, could enhance learning stability and promote more robust strategic behaviour in multi-agent competitive settings by continuously adapting to evolving policies. Additionally, a modular architecture, similar to the phase-specific design proposed by Blomqvist [13], could be employed, in which separate policy networks are assigned to the drafting, attacking, and fortification phases. This decomposition reduces interference between distinct decision contexts and allows each component to specialise within a narrower action space, potentially resulting in more stable training dynamics and more coherent strategic behaviour.

6.2.2 Extensions to Game Mechanics

For tractability and experimental clarity, several aspects of the full Risk ruleset were constrained. While these simplifications enabled controlled analysis, relaxing them would allow for a richer and more realistic strategic landscape. In particular, the introduction of alliance dynamics would incorporate the psychological and diplomatic dimensions central to human play. Similarly, exploring non-blitz combat mechanics would alter risk-reward calculations within engagements, potentially changing optimal aggression levels. Finally, modelling more nuanced trading tactics for territory cards could introduce additional layers of long-term planning and temporal optimisation. Extending the environment to accommodate these elements would move the framework closer to the full strategic depth of the original game.

Chapter 7

Legal, Social, Ethical and Professional Issues

7.1 Legal Issues

This project does not use any external data. All data used to train and evaluate the reinforcement learning models was generated internally and curated by the author. No personal or sensitive data was collected from external sources, ensuring compliance with GDPR and related data protection legislation.

All code for this project was written entirely by the author. The trustworthiness and reliability of all third-party dependencies were considered before use, and the **risk-dice** repository created by SMG Studio was leveraged only to generate the Risk probability matrix. All dependencies have been employed in accordance with their respective licenses, and the project is intended solely for research and educational purposes, with no commercial distribution.

7.2 Social and Ethical Issues

The training of reinforcement learning models can be computationally intensive and therefore may have an environmental impact due to energy consumption. In this project, care was taken to conduct model training efficiently, including limiting the number of training runs, using optimised code, and performing all computation locally rather than on large-scale cloud resources. The author acknowledges in the "Future Work" section that computational requirements are a fundamental aspect of reinforcement learning and that similar tasks often rely on substantial server resources. Nonetheless, future AI research and extensions of this project should continue to adopt responsible practices, taking precautions to minimise environmental impact as reflected in the design choices made in this project.

7.3 Professional Issues

This project has been conducted in accordance with the four key principles of the BCS Code of Conduct [34]:

- 1. Public Interest:** This project ensures that all work is conducted with regard for public well-being. No external data was collected, and all model evaluation was performed internally, avoiding any risk to users or the public.
- 2. Professional Competence and Integrity:** The implementation was developed independently following established software engineering and research best practices. The results are presented transparently and follow standard scientific methodology, with clear instructions provided to reproduce all experiments, confirming the validity of the findings.
- 3. Duty to Relevant Authority:** This project adheres to legal and institutional guidelines, including respect for intellectual property, licensing requirements, and research ethics, ensuring compliance with relevant authorities and regulations.
- 4. Duty to the Profession:** Throughout the project, professional standards in AI development and academic research were maintained. Consideration was given to sustainability, responsible experimentation, and ensuring the work remains appropriate for research and educational use only.

References

- [1] Wikipedia contributors. Nondeterministic algorithm, 2025. URL https://en.wikipedia.org/wiki/Nondeterministic_algorithm.
- [2] Hasbro. Risk game rules, 2016. URL https://instructions.hasbro.com/api/download/B7404_en-us_Risk-Game.pdf.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [5] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015.
- [6] Amanatullah. Vanishing gradient problem in deep learning: Understanding, intuition, and solutions, 2023. URL <https://medium.com/@amanatulla1606/da90ef4ecb54>.
- [7] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [8] GeeksforGeeks. Bias-variance trade-off in machine learning, 2024. URL <https://www.geeksforgeeks.org/machine-learning/ml-bias-variance-trade-off/>.
- [9] Wang Chi Cheung. Exploration-exploitation trade-off in reinforcement learning on online markov decision processes with global concave rewards. *arXiv preprint arXiv:1905.06466*, 2019.
- [10] Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 2012.

- [11] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 2016. URL <http://www.nature.com/nature/journal/v529/n7587/full/nature16961.html>.
- [12] Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 2017.
- [13] Erik Blomqvist. Playing the game of risk with an alphazero agent. Master’s thesis, KTH Royal Institute of Technology, 2020.
- [14] Andries E. Brouwer. Length of a go game, 2003. URL <https://homepages.cwi.nl/~aeb/go/misc/gostat.html>.
- [15] Christoffer Limér, Erik Kalmér, and Mika Cohen. Monte carlo tree search for risk, 2020.
- [16] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [17] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4): 279–292, 1992.
- [18] Linus Strömberg and Viktor Lind. Board game ai using reinforcement learning, 2022.
- [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [20] Cameron R. Wolfe. Proximal policy optimization for llms, 2023. URL <https://cameronrwolfe.substack.com/p/ppo-llm>.
- [21] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171*, 2020.
- [22] stable baselines3. Maskable ppo — stable baselines3-contrib, n.d. URL https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html.
- [23] RiskOdds. Risk odds calculator, 2024. URL <https://riskodds.com/>.

- [24] J. A. Osborn. Markov chains for the game of risk, 2001. URL https://web.archive.org/web/20141113210920id_/http://www4.stat.ncsu.edu:80/~jaosborn/research/RISK.pdf.
- [25] SMG Studio. Risk dice, 2017. URL <https://github.com/smgstudio/risk-dice>.
- [26] cmglee and Gr0gmint. Risk board.svg, 2021. URL <https://commons.wikimedia.org/w/index.php?curid=106498695>. Licensed under CC BY-SA 4.0.
- [27] Baeldung. Check if two nodes are connected, 2021. URL <https://www.baeldung.com/cs/check-if-two-nodes-are-connected>.
- [28] Refactoring.Guru. Strategy design pattern, 2026. URL <https://refactoring.guru/design-patterns/strategy>.
- [29] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2025.
- [30] Reinhard Diestel. *Graph Theory*, volume 173. Springer-Verlag, Heidelberg, 2017.
- [31] Fiona Middleton. The 4 types of validity, types, definitions & examples, 2022. URL <https://www.scribbr.co.uk/research-methods/validity-types/>.
- [32] Neil Robertson, Daniel Sanders, Paul Seymour, and Robin Thomas. The four-colour theorem. *Journal of Combinatorial Theory, Series B*, 1997. URL <https://www.sciencedirect.com/science/article/pii/S0095895697917500>.
- [33] GeeksforGeeks. Bernoulli trials and binomial distribution, 2026. URL <https://www.geeksforgeeks.org/maths/bernoulli-trials-binomial-distribution/>.
- [34] The Chartered Institute for IT BCS. Bcs code of conduct for members, 2022. URL <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>.